

OCR GCE A

COMPUTER SCIENCE

PROJECT

H446-03

Name : Anodhya Lankathilaka

Candidate Number : <INSERT CANDIDATE NUMBER>

Institution Name : Hatch End High School

Title of Project : Password Vault (SOFTWARE)

H446-03 – PROJECT CONTENTS

TABLE OF CONTENTS

A. Analysis.....	4
Problem Overview	4
Stakeholders	6
Existing solutions	7
Essential Features In detailed	10
Limitations of proposed solution	13
Requirement gathering	14
Success Criteria	16
B. Design	19
systemeatic problem decomposition	19
Detailed design structure	25
Database design (ERD)	31
usability features	33
Key variables, data structure and classes	35
Test data (White box testing)	36
test data for iterative development (Black box)	40
further data for post-development phase	45
C. Developing the coded solution (“The development story”).....	46
Development of FRONT END	48
FRONT-END test table:	55
Development of Back End	57
17/11/2025	57
22/11/2025	59
28/11/2025	63

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>	
30/11/2025	68
2/12/2025	72
7/12/2025	75
8/12/2025	76
13/12/2025	80
D. Evaluation	90
Project Appendixes	92
Refference	93

A. ANALYSIS

PROBLEM OVERVIEW

Project title: The Password Vault.

Problem: In an era where almost everything is done in online through internet, most people find it hard to remember credentials (such as passwords, email, and username ...) for all the sites they sign in. Even though, people find a place to store their credentials virtually or physically, the biggest problems that arise is, “is it actually safe? Could a hacker or a person with negative intentions get access to the credentials?”

Solution: To solve this problem I have come up with a software called password vault which allow anybody to create an account and store their credentials in a local database. To tackle hacking, the software will encrypt the credentials with a custom algorithm and save the data in a database, so that the actual credentials will not be visible (already modified to another form). And a Master password is created by the user while creating account to ensure that only he/she can see and modify the credentials they saved in the vault. This will encourage users to save their credentials in their devices securely.

Features:

1. Create a **Master password** when the user creates the account, so that only the user can view their credentials and modify if needed. **Validation test** will be conducted as the Master password is created so that it is strong enough to be used as the key for the entrance to view saved info.
2. **Custom algorithm** with the use of a formula will generate a modified password which will be saved in the SQL database (encrypting), and the encrypted credentials will be

reverse engineered with the **reverse formula** whenever the user needs to view the credentials.

3. The user will be able to **categories the credentials** they are about to save so that it will be easier for the person to filter out when they search for the credentials they would like to view. Would use different colours for each category to **abstract** the grouping.
4. A **password strength checker** will be implemented and will be shown under every saved credential to inform or warn users whether the credentials being saved has a higher chance of being hacked or its strong enough. The level of strength will be shown with a **colour flag** (red flag: must strengthen the credentials, yellow flag: medium strength and green flag: completely fine with the strength) instead of a **statistical measurement** (percentage value) to reduce complexity and motivate users to improve passwords if needed.
5. Software will allow user to manually create a password as he/she wishes, and an **Auto Password generator** option is available for the user if he needs to create the password automatically with high security.

Computational Thinking:

Algorithmic thinking: A custom algorithm will be developed to encrypt and store the user credentials in the SQL database so that hacking the database would not let a third party to access passwords and usernames. This algorithm will be reversed whenever the user needs to access the password. Apart from encryption, another algorithm will be run for each credential to check the strength of the password and represent the status with flags as mentioned in the features. Finally, an algorithm will check for validation when creating the **Master password** where it should meet certain requirements.

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

Abstraction: The encryption process will not be shown for the user, once the user needs to save credentials, he will be shown a simple and clear form that enables to input the credential and click the save button. Next, the strength checker will only output the status by a word and flag representation so that it highlights the status clearly without showing any complicated numeric or processes happening behind the code.

Automation: An Auto password generating option will be available for the user to generate a password from a custom algorithm so that for important site they visit and are unsure about any password they manually input, this might be the safest next available option. Users who already created an account will be logged in if the access credentials are matched with the saved credentials in the database.

STAKEHOLDERS

End Users:

End users are anybody who are willing to use this software to store their credentials. The End users can be at any age without any age limitation. The main purpose they use this software is to store their credentials safely so that they don't need to worry about forgetting the access information for various sites over time as most humans do.

Needs that satisfy the End Users:

- **Security:** This software provides high security for data. Users store credentials in the software database with encryption, Master passwords (Authentications) to log in to view stored data. Any hacker or third party must pass these phases to access the saved passwords which is possible but less possibility. For further improvements threat mitigation actions such as Firewalls, powerful servers and antivirus software could be allocated.
- **Simplicity:** Abstraction will be used on each GUI page to build a simple and clear software. This will help the users at any age (especial focus to the group of user who are

aged above 60) to identify what they need to be done without help from another person.

Quick error reflection messages will be popped during validation tests to overcome mistakes and maintain a clear environment for users.

Most user needs are covered by me, by looking at the problem in user's perspective. However, during a conversation with my colleagues about the project, they came up with the idea of indicating the strength of the password with both a text and a color (For example: Text: "Strong" colored circle: "Green"). These colleagues were psychology students and according to their justification, human brains identify colors and images faster than letters. And this is the reason I decided to include different colors for each category as well, so that as users use the software for long time, they can recognize the categories without reading the category name.

What is expected from End Users: Every End User must have an account created to use the resource. A stronger password satisfying the validation requirements should be created for the Master password so that this key which allows the user to access any credential they save is only known by that individual user. Finally, every user must change passwords that are not strong enough according to the strength checker measurements, the software will not force to change the password, but users must be aware of the danger of saving weak credentials.

EXISTING SOLUTIONS

1. **Bit warden:**

This is an open source password saving software that has both offline and cloud encryptions to save individual credentials.

Strengths:

- Include powerful passwords generators and strength tester.
- Widely supported on desktop, mobile and browsers
- Include End-to-end encryption, those credentials are secure even on the cloud

Weakness:

- Two Factor Authentication (2FA) requires a premium subscription
- This software is primarily cloud-based by default.

This software uses both auto password generator and strength checker which I will be developing for my project.

2. **KeePassXC:**

This is a fully offline desktop focused software that saves user credentials by encryption.

Strengths:

- Open source and free (User perspective).
- Support categorization, with powerful password generators and browser integration.
- Since Open source, this software has a group of community with free services.
- Fully offline makes it more secure as it is not part of the internet.

Weakness:

- This software will be complex for beginners to configure syncing.
- Source code is open for public which is not secure.

This software contains the categorizing feature that I will be developing for my Password Vault project, apart from categorization, this also contains the Auto password generator. Being fully offline and less complex makes it similar to my project of the Password Vault.

3. **Enpass:**

This is a freemium software that allows users to use with zero cost of purchase for the basic use, however, more advance features and content are available to be purchased for use.

Strengths:

- Uses local encryption via SQLCipher, without any central cloud.
- Include auto password generator and strength checkers

Weaknesses:

- Mobile version is free only for 25 saving credentials, more than 25 needs purchase
- The Graphical User Interface is less user friendly and more traditional looking.

This software contains the auto password generator and the strength checker which I will be developing for my project. This software can be used for free for a certain limit. However, the Password Vault that I develop is completely a free software that has no limit for saving credentials.

4. KeeWeb:

This is a cross platform for both desktop and web GUI versions, meanwhile this software is an Open source software with local file handling features (no server required).

Strengths:

- Auto password generator is available for desktop version
- Highly secure with encryption

Weakness:

- Open source software, and therefore might have a slight chance for modified versions.
- The files are more advanced making it less compatible.

This software does not have strength checkers and password categorization which I will be developing in my software. However, KeeWeb does support in multiple cross

platforms and have good security compared to other Open source password saving software.

ESSENTIAL FEATURES IN DETAILED

1. Validation:

This is the first most essential feature the user encounter when he creates an account with a unique user id and a Master password (the key to unlock saved credentials). Master password must be impossible to crack by a third party as it is the most essential component in the software, to make sure it is secure, the software user must pass through validation tests.

Validation test:

- a) At least 8 characters long.
- b) 8 characters must contain at least one:
 - Uppercase letter (A-Z)
 - Special character (! \$%^&...)

An error message will be shown to the user if the Master password they create does not meet the validation requirements and the create button will only be available if requirements are met. This is done for both the user and software safety, to mitigate threats from third party access control.

2. Encryption:

Most of the time, hackers tend to attack the database where all the information are stored rather than crashing the whole software and making awareness that somebody is try to hack the software. To mitigate this threat, a custom algorithm

will be run to encrypt the credentials a user saves in the database, so that even if a hacker accessed the database, the information that is visible will be modified to another fake version.

This is the most suitable and best option to mitigate such threat. However, for further improvements, a firewall to only allow trusted IP addresses, multi-factor authentication can be implemented.

Process of Encryption: The algorithm contains all the characters in the keyboard that user can use to create a password. Next, each character is saved in an array/list so that each character can be accessed by a certain index number. A custom build formula will be used to generate the modified credentials, by considering the array as an end to end connected circle. Finally, the reverse formula will be used when the user needs to see the password. This is considered the most complex and time consuming section in the whole project since it carries the most important role of the Password Vault.

3. Categorization:

This is a new feature for that password vault which is unique compare to most of other password saving software available. This feature is mainly used to filter out credential saved by users so that it is easier to search. For example, if the user has already saved 50+ credentials, it would be hard to search for the credentials he is looking for since all the passwords are saved in the same GUI design. Therefore, to tackle this problem, the user will be able to choose which category the credentials are belong to and then save the credential in that category which will be displayed in different colors. For example, Finance category will be saved in yellow, Medical category in blue and etc.

This is a physiological approach as human remembers colors and images within a short time period compared to letters and numbers. This method will help them filter the credential details they are looking for much faster.

4. Password strength checker:

This feature runs through an algorithm to test level of strength of the saved credentials (passwords).

Algorithm description: A numerical scaling method will be used in the test. Two tests will be conducted each with different strength checks:

- 1) **Length:** The longer the password, the stronger it will be. Generally, a password that has 10+ characters is considered to be strong.
- 2) **Character variety:** This test will look for mixture of character types, For example, Uppercase letters (A-Z), lowercase letters (a-z), Numbers from 0-9, and finally special characters (% ^ & * ...).

These two tests will each result a value from 1-3, tests resulting in three signals that the password is strong in that particular factor (length, character variety and ect...).

Next the sum of the two tests will be calculated and ranged as Maximum value as 6 and Minimum value as 2. This boundary will be used to scale the final overall result which will be shown to the user.

Strength Scale:

- 2 WEAK
- 3 - 4 MEDIUM STRENGTH
- 5 - 6 STRONG

5. Auto Password generator:

This feature is available for user in all password saving pages. This option allows users to automatically generate a strong password within seconds. How does the algorithm work? Firstly, the user will be asked for the length of the password he is willing to have. The length will be from 8 – 16 to make sure the password will meet a minimum safe length. A pool of all character will be saved in a list in the code. Characters will be chosen randomly from the pool until the length is met. To make sure that at least one character is used from the four character sets (Uppercase, lowercase, numbers and special characters), a forced round will be

round will be run at the beginning to have one from each character set, next the rest will be chosen at random from the pool as mention above.

LIMITATIONS OF PROPOSED SOLUTION

1) Algorithm:

The solution provided limits a third party access, and even if a third party accessed the system, the data in the database will be modified from the custom encryption algorithm used in the software. However, the algorithm is the key for a hacker to hack all the data stored in the database. If a third party access the code of the software, he can view the custom formula that is used to encrypt the saved credentials and reverse engineer to gather all the saved passwords in the system. A secure encryption technique such as ASE (Advance Encryption Standards) or RSA can be developed for future use.

2) Mater password:

Master password is the key for the user to view his saved passwords, and this will only be known by the user. However, in reality, this can be a target of a person with negative intensions to hack the Master password and have a copy of the key with him. Even though the Master password is hashed into the database, there are countless ways for hackers to hack the data. The security can be increased using powerful servers, firewalls mitigating the chance for third party attacks such as SQL injections, DOS, Buffer overflow and etc. Also, there will be no solution for users forgetting the Master Password

3) Auto password generator:

The auto password generator will be built with the random library in python and a certain function that will be run to output a random password whenever the customer clicks the “Auto password generator”. However, the passwords will

follow the same rules such as 8 characters, 1 special character, 1 Uppercase. This will lead to predictable passwords if user over use this feature. Including AI to help build random passwords without standards could be more efficient and secure compared to the traditional random password generating methods.

4) Multi device synchronization

This software limits the user only for the device which I developed, because this is a standalone desktop app. The users cannot sync data to other devices as I use a local database to store user passwords. To overcome this limitation, a real life solution would be having a central sever with shared database such as MySQL or PostgreSQL in the cloud, or an online API that all devices connect to.

REQUIREMENT GATHERING

1) Hardware:

- **Storage space:** A minimum of 50MB should be available for the database to store database files and future updates with all data storing capacity for 500+ credentials. This space should also be enough to store the program code too.
- **RAM:** Random Access Memory, with a minimal requirement of 4GB should be used to ensure smooth execution of the program, especially when using GUI features from Tkinter or dealing with multiple records in the database.
- **Desktop computer or Laptop:** All the hardware devices in a computer (input and output devices) is needed for both the developer and the users of the software system. A special attention for the processor, it is better to have a dual core processor to efficiently run the python interpreter and handle the encryption.

2) Software:

- **Operating system:** Python and SQLite are cross platforms, so the software will work on all major Operating Systems such as Windows, Linux and macOS. The project will be developed in Windows Operating System as it provides a wide compatibility with Python tools and offers a user-friendly environment for development.
- **Python Interpreter:** A Python 3.8 or higher interpreter is required to run the program. The project uses features such as f-strings, modern random modules and built in SQLite support, which are fully supported in recent versions of Python.
- **SQLite:** The software uses SQLite, which is built into Python via the SQLite3 module. This provides a simple local file-based database system, that eliminates the need for a separate database server. This software is chosen due to its lightweight nature and ease of setup.
- **IDE:** The program is developed using PyCharm as the IDE to write the Python code of the Password Vault. And Tkinter will be used to build the Graphical User Interface. Since we write the whole project using Python, both the software provides powerful but clear workspace

SUCCESS CRITERIA

No.	Requirement	Justification	Success
1	Ability to log-in to the software by clicking “Login” button	The user must be taken to Login page where he can input his user id and Master password to log into the system	
2	Ability to create a new account by clicking “Create account” button	The user must be able to create a non-existing account by creating new User id and Master password	
3	Ability to create Master Password	The user must be able to input any character as he wish but meeting validation requirements	
4	“Save” button to begin validation test	An error message must pop up if even a single validation rule out of three is not met.	
5	Hashing the Master password into the database	Master password must be hashed and saved in the database table.	
6	Log-in page after creating account	If successfully created an account, he must be asked to enter login details to take him to Main page.	
7	Categories being clear and visible	Each category must have a different color and able to click as “buttons” for user to navigate into that specific category	

8	Functionality of the “+” symbol button on bottom right	Clicking the plus symbol must navigate user to the page dedicated to store Passwords	
9	Does the Dropdown list show all the possible categories	The category dropdown list must show all the developed categories as the user click the down arrow symbol	
10	Input box for reference name must store a name	The user can only input letters and numbers in this input box as there is no need for special characters.	
11	Input box for the password stores any character for any length	The user is able to input the password as they wish using any characters with no length limits.	
12	“Auto generate” button to generate a random password	User must be able to click the button and a random password should be displayed on the input box for the password	
13	“Save” button on bottom right	Clicking the save button must begin the validation for every inputs in the page. If user does not meet validation rules, error message must be displayed.	
14	Encryption of the password	Once input validation rule are met, the password must be encrypted and automatically saved into the database table.	
15	“Cancel” button available for user to decline the password saving process	If user click the “Cancel” button, he must be taken into the main page again.	
16	Each Category page shows the passwords saved under that category	Clicking a Category displays all the passwords saved under that category.	

17	“View” button enable user to view the saved data	Clicking this button should take the user to the Authentication page where Master password will be required to enter	
18	Authentication page	To access the data, user must input the Master password into the input box provided.	
		Clicking “Enter” button will hash the input text and compare with the hashed actual Master Password.	
19	Decryption process decrypt the stored data	The stored data being automatically decrypted as soon as the authentication is successful	
20	Display password details as authentication becomes successful	All the data the user search will be displayed clearly and accurately	
21	“Change password” option available as an input box	The input box will act same as the password storing input box the user met earlier to store the password	
22	“Change” button option to start process of changing the password	Password undergo the encryption algorithm and save the new password in the same database location the previous password was stored	
23	Strength checker display a result of the strength of that certain password	Strength checker function automatically process and display under the password details user view.	
24	“Back” button available to navigate pervious pages	This button takes the user to the Main page as the user is done checking saved password	

--	--	--	--

B. DESIGN

SYSTEMEATIC PROBLEM DECOMPOSITION

1. Log in

Users who already created their account can log in to the system with their access credentials. The access credentials are User ID (which is a string that have letters and numbers, and every user must have a unique id for themselves) and Master password (Which is the key for accessing all the saved passwords). The user will not be taken to the main page until they enter their correct log in details. However, any user hasn't created the account must create the account first.

The image shows a mobile application interface for a 'Password vault'. The screen has a light purple background. At the top, there is an orange header bar with the text 'Password vault' in white. Below the header, there is a white rectangular area containing the login form. The form is titled 'Login' in bold. It has two input fields: the first is labeled 'User ID' and the second is labeled 'Master password'. Below these fields, there are two buttons: 'Back' on the left and 'Enter' on the right.

Figure 1 Log in page

2. Create account

This page is developed for users to create an account. The First input box is for user to create a User ID that does not exist in the system. If user enter an already existing user id, an error message will be shown to create another User ID.

Second input box is for user to create a Unique Master password. Once again, the same apply for Master password, duplicate Master passwords are not allowed. Each individual must have a unique ID and Master password.

The image shows a mobile application interface for creating an account. At the top is an orange header with the text "Password vault". Below this is a light purple box containing the "Create account" section. Inside this box, there is a title "Create account" followed by a red asterisk. Below the title are two input fields: "Enter User ID here" and "Enter Master password here". Under the password field, there are three lines of text indicating password requirements: "At least 8 characters", "At least 1 Special character", and "At least 1 Uppercase letter". At the bottom of the purple box are two buttons: "Back" and "Save".

Figure 2 Create account page view

3. Auto password generator

All users have this feature available as an option every time they start the process of storing passwords in the software database. A simple click on a button specifically

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

created to allow generate automatic passwords will help users to create a password within seconds. Steps to generate a password are as below:

- 1) Random generation
 - Once user click the auto password generator, a password will be shown on the screen, with a random length between 9 to 15 random characters' types (numbers, Uppercase and Lowercase letters).
- 2) Acceptance or recreation
 - The user can regenerate passwords by clicking Auto generate button until he is satisfied with the password, or click save to store the generated password shown on the screen. The Auto generate button will loop the same process of creating a random password until the user is does not click it again.

The image shows a mobile application interface titled "Password vault". It features a white background with a light purple border. At the top, there is an orange header bar with the text "Password vault" in white. Below the header, there are three input fields: "Category" with a dropdown arrow, "Reference Name" with a red dot at the end, and "Password" with a red dot at the end. Below these fields are three buttons: "Auto generate" in the center, and "Back" and "Save" at the bottom left and right respectively.

Figure 3 Password saving and Auto password generator

4. Password encryption

Password encryption is a sub routine that is coded in the python code and being called each time a user saves a password. This sub routine will help the user password to securely rest on the database.

- A sub routine will be created know as encryption. This custom build encryption method will have the whole set of characters in an array, next a mathematical formula will be used to assign each character of the actual password another character. I consider the characters as a never ending circle.

This encryption is not the most efficient and strongest in real life Password saving software. However, I chose this custom encryption to demonstrate how encryption originated to the world.

5. Categorization of credentials

As Password vault will save hundreds of different passwords. I decided to categorize each password into different sections so that it will be easier for the user to search password they stored in past. The user must input the category when saving the password through a drop down list. Some main categories would be Medical, Financial, Education and etc. A simple design of how categorization will help users to search quick is as below

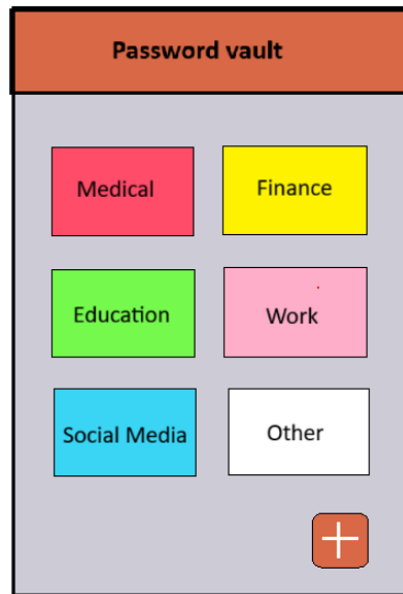


Figure 4 Mobile view of categorization feature

By clicking any category, the user will be taken to the passwords saved under that specific category.

6. Password strength checker

This feature will be seen by the user when he views the saved passwords. The strength status will be shown under the saved passwords and this status include words and a colored sign to inform the user about how strong the password he/she have saved previously.

To design this feature, a sub-routine called Strength checker will be called every time the user access to the saved password. This sub-routine runs only for the password the user view so that every password status is not being saved on the database, since it is inefficient. The password strength checker will look as below:

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

The screenshot shows a mobile application interface titled "Password vault" in an orange header. Below the header, the user's name "Patches" and password "Alex@1997" are displayed. A text input field contains the placeholder "Change password here". Below the input field is a "Change" button. At the bottom, the "Strength status: STRONG" is shown next to a green circle icon, and a "Back" button is located at the very bottom.

Figure 5 Strength checker

Strength checker results: A strong password will be given “Strong” with green circle. An average strong password with slight chance of being crack by any other party will be given a “Mid Strong” with orange circle. And finally a weak password that has very high chance of being hacked will be given “Weak” with red circle. This will clearly inform the user the password status both visually and verbally.

DETAILED DESIGN STRUCTURE

User view process

Below picture shows a design of the whole software (password vault) which the user will be seen when they use. Each stage will be given explained in detail showing the whole workflow of the software system.

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

The figure displays eight mobile app screens for a 'Password vault' application, arranged in two rows of four. Each screen has an orange header with the text 'Password vault'.

- Screen 1 (Top Left):** Features a 'Login' button, the word 'or', and a 'Create account' button.
- Screen 2 (Top Middle-Left):** Titled 'Login', it contains input fields for 'User ID' and 'Master password', with 'Back' and 'Enter' buttons at the bottom.
- Screen 3 (Top Middle-Right):** Titled 'Create account', it includes input fields for 'Enter User ID here' and 'Enter Master password here'. Below these are password requirements: 'At least 8 characters', 'At least 1 Special character', and 'At least 1 Uppercase letter'. 'Back' and 'Save' buttons are at the bottom.
- Screen 4 (Top Right):** Displays a grid of category buttons: 'Medical' (red), 'Finance' (yellow), 'Education' (green), 'Work' (pink), 'Social Media' (blue), and 'Other' (white). A red '+' button is in the bottom right corner.
- Screen 5 (Bottom Left):** Contains a 'Category' dropdown menu, input fields for 'Reference Name' and 'Password', an 'Auto generate' button, and 'Back' and 'Save' buttons at the bottom.
- Screen 6 (Bottom Middle-Left):** Titled 'Medical', it shows three entries, each with 'Name' and 'Password' fields and a 'View' button: 'Patches', 'Belmont health', and 'X ray lab'.
- Screen 7 (Bottom Middle-Right):** Titled 'Master password', it has a single password input field and 'Back' and 'Enter' buttons at the bottom.
- Screen 8 (Bottom Right):** Displays 'Name: Patches' and 'Password: Alex@1997'. It includes a 'Change password here' input field, a 'Change' button, and a 'Strength status: STRONG' indicator with a green circle. A 'Back' button is at the bottom.

Figure 6 Full workflow design

Stage 1:

The first stage of the software is to check if the user has an existing account or not. The user can enter log in details and enter to the system if they already created an account. If not, the user must first create an account and then try log in with those credentials.

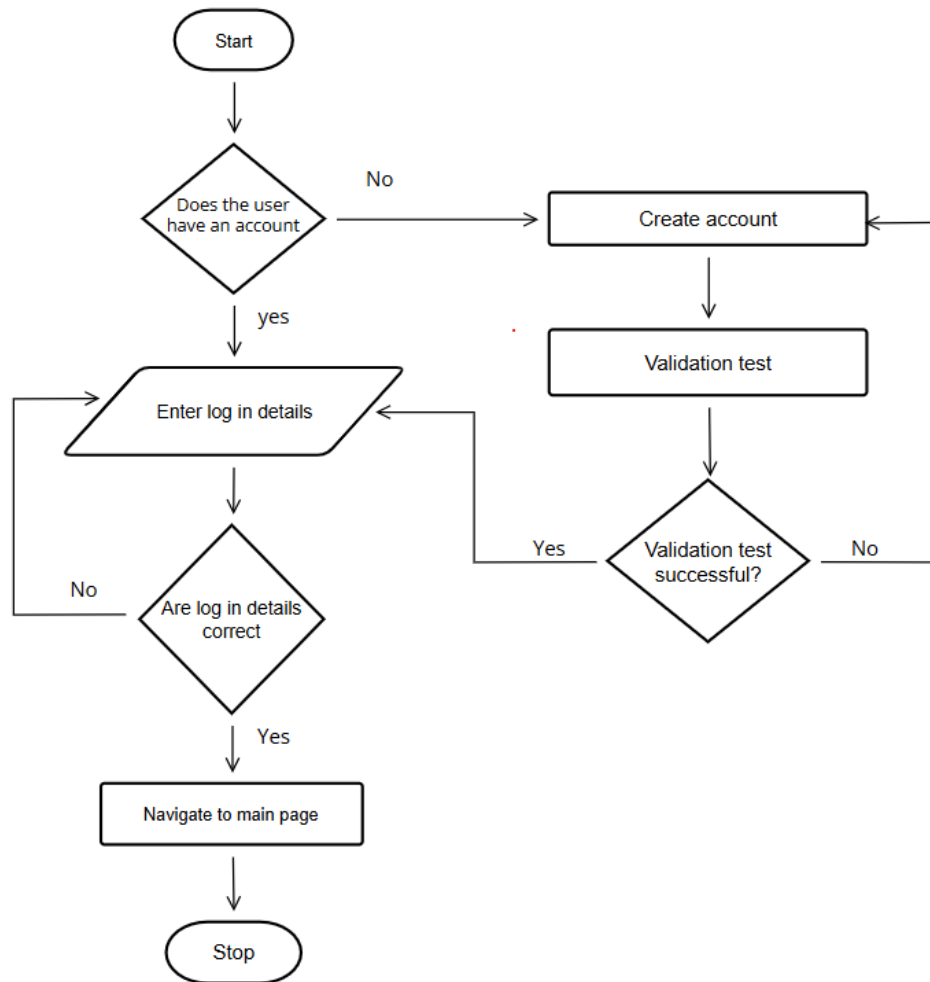


Figure 7 Workflow of Stage 1

Stage 2:

The plus symbol at the bottom in the main page (will allow the user to store passwords which would be stage 3). There are 8 different key categories in the software (even though the picture shows 4 categories on stage 3). They are:

- **Financial** – banking, mobile wallet, investment apps
- **Medical** – Pharmacy accounts, hospital accounts, health insurance
- **Education** – School, university portals, e-learning sites

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

- **Work** – Office accounts, work place tools
- **Social Media** – Instagram, Facebook, Twitter
- **Entertainment** – Netflix, Spotify, BBC
- **Shopping** – Amazon, e-bay, Waitrose
- **Other** – Traveling details, hotel, airline, Other apps and accounts

This categorizing is done by the user every time he saves a password (not done by AI or any other automatic process). The user does this categorization in stage 3 and is completely on his hand to allocate the password to the correct category.

Stage 3:

As shown in the picture, the user must enter the category from the dropdown list. A text bar for user to input a meaningful name and strong password will be developed. Also, there will be an auto password generate button, where a sub routine known as “Random password” will be called every time the user click the button. This sub routine will perform a set of code that will return a new random password on to the password text bar. This subroutine will always provide a random and strong password with no user involvement. Two buttons will be available, where “Save” button will encrypt the password and save it in the database (and this process is not visible by the user. Finally, the “Cancel” button will simply take the user back to the main page stage 2)

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

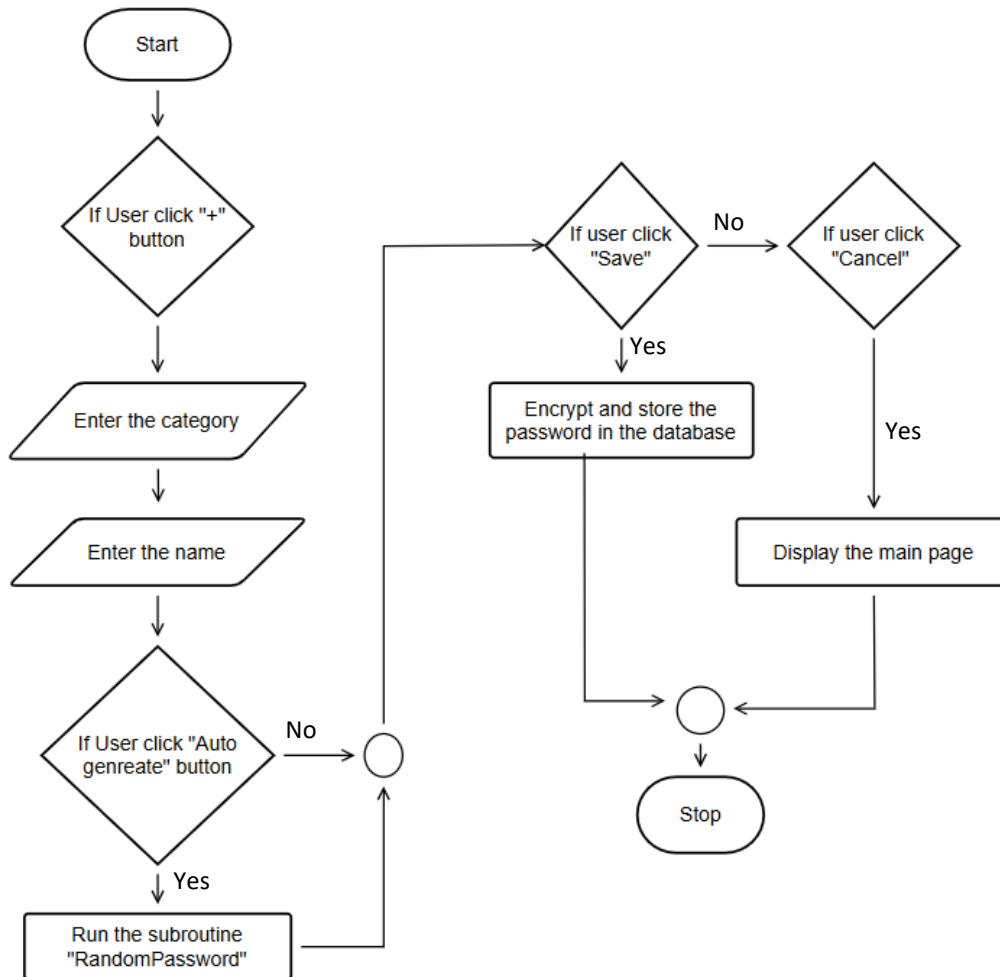


Figure 8 Workflow of stage 3

Stage 4:

According to the above picture, stage 4 is an example of what the user will be seen in medical section after saving multiple passwords under the category of medical. A simple display box will be given to each password, and this display box will only show the name and a hidden password with the symbol of “ * ”, whenever the user click one “View” button, he will be taken to stage 5.

Stage 5:

This stage is a security stage where the user needs to input the Master Password into the text bar provided as shown. Each time the user enters the Master password, it will be converted to the hash and compared with the Master Password the user build at the beginning (actual Master password) which is hashed and already stored in the database. If the Master Password user entered in stage 5 does not match, user will need to try again. The same process will be done until user enter the actual Master Password.

Stage 6:

Once the user passes through stage 5, the name and password will be shown. The decryption process of the saved password will be done internally with no user involvement or visibility. Meanwhile, there will be a text bar: as shown on the workflow design stage 6 where the user could change the password if needed. This option is available only on stage 6 due to security measurements. Once the user input a new password on the text bar and click the button “change”, the password in the database will be changed.

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

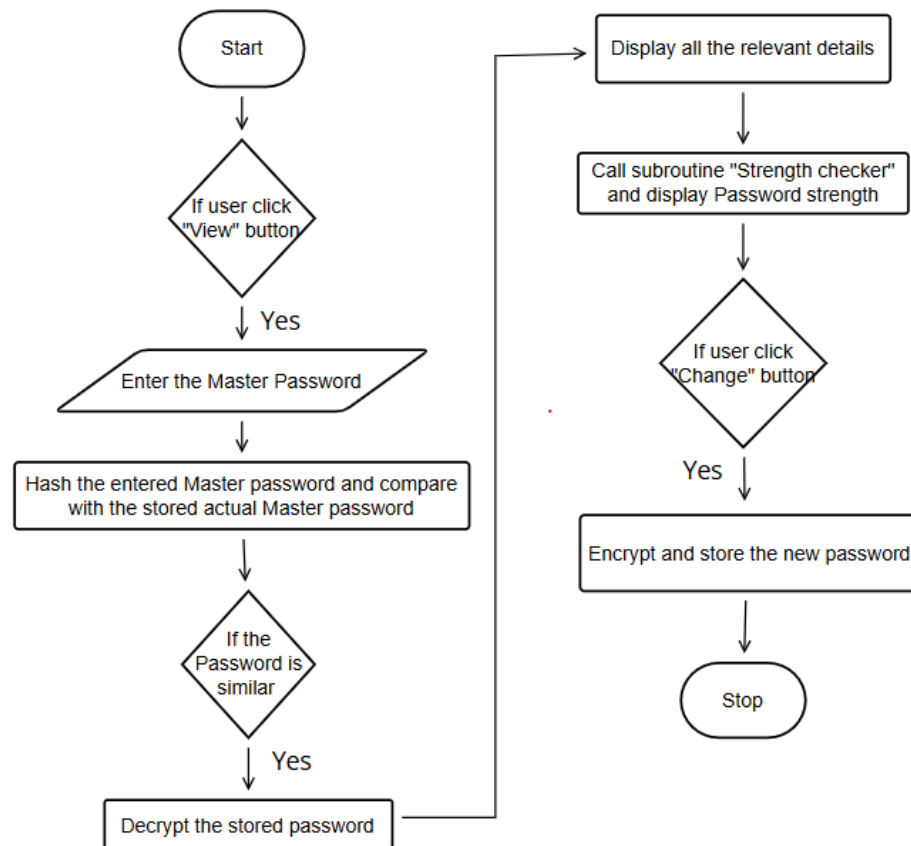
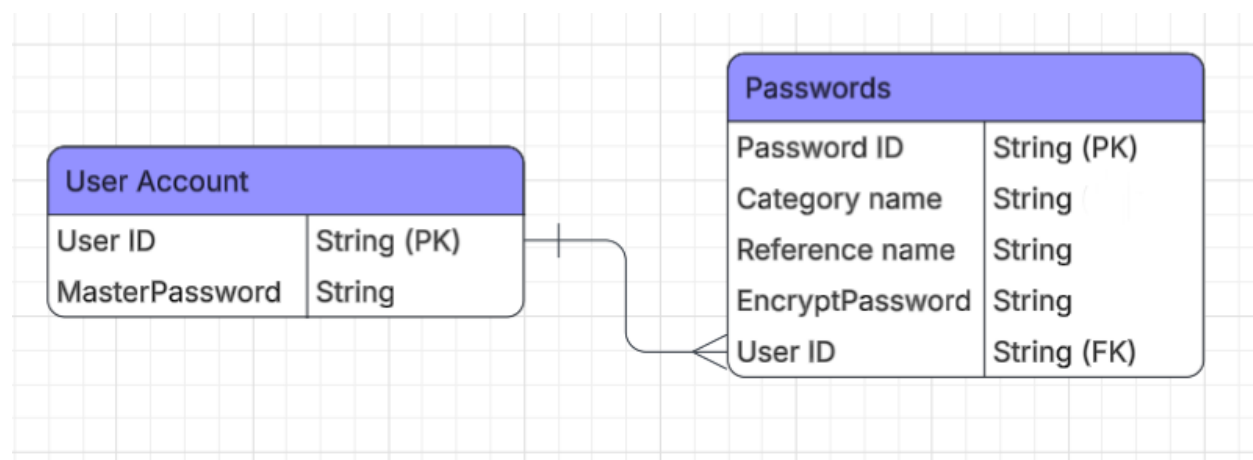


Figure 9 Stage 5 and stage 6 workflow

DATABASE DESIGN (ERD)



Full functional flowchart of the software

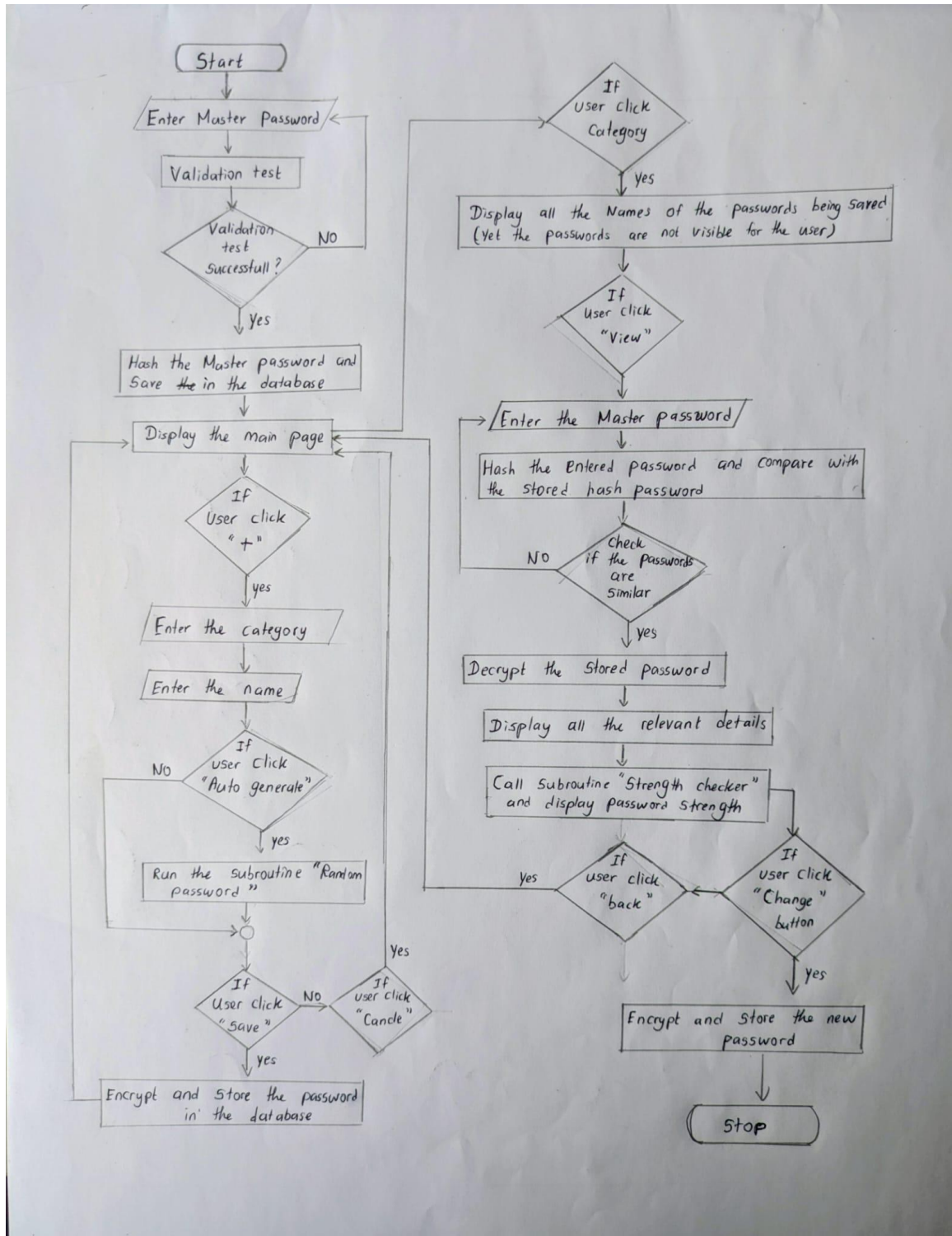


Figure 10 Full functional workflow of the system

USABILITY FEATURES

As shown in figure 5, The user interface is developed with simple interface elements, no complex user requirements are available throughout the whole system. All complex processes such as Master Password hashing, Password encryption and decryption, Auto password generation and Password strength checker are hidden and run internally. The three main color combinations used in the software system is as below:

- #D96846 [Orange color] 
- #CDCBD6 [Light Grey color] 
- #FFFFFF [White color] 

In the Master Password creation [[Figure 2 Create account page view](#)] the page will be developed simply with a single Text bar and Validation rules of the Test undergo each time the user press the button save. These validation rules are in the bullet form and is stated clearly for any user. Once the user successfully created the Master Password, he will be taken into the main page. The user will see all 9 key categories as mentioned above in the topic. Each of the category will be built with different color to make it more user-friendly[[Figure 4 Mobile view of categorization feature](#)]
In modern days, most software uses the Plus symbol to represent the creation of new item. Therefore, I decided to include the Plus symbol to function the process of storing password to keep it simple and clear.

Next, the user interface for the process of storing a password [[Figure 3 Password saving and Auto password generator](#)] is created with a simple dropdown list for user to enter the category of the password, two text boxes for name and Password. Again, the “Auto Generate” button hides the process of randomly generating a password to not make user complicated. The user also can manually add a password. I decided not to have two different buttons for Auto generate and Manual generate by user to reduce complexity.

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

User can view the password by simply clicking the view button, and a simple user interface will be shown for user to enter the Master Password (which is a high security measure used in this software system to allow only the user to view the saved credentials).

Finally, the user can view the details and modify the saved passwords if he successfully passed through the Master Password security measure. For password modification, a text bar will be built under the password details. The strength status of the password user view will be shown at the bottom with a color and a small text [Figure 5 Strength checker]

1) Offline access

- This is an offline software allowing users to access the software and use it without internet.

2) Security

- Master Password is generated and acts as the key to access all the saved passwords. The Master password will only be known by the one who created it (user) and will be hashed in to the database.

3) User friendly interface

- As shown above each page is built with a simple logic and clear graphic design to help user of any age to use the software without guidance. The default language is “English” and the color combinations user are eye catching.

4) Password generation

- Click of a single button will generate a random password for users who are unable to think of a new password. This will help user save valuable time in busy moments and the user can continue clicking the button until he is satisfied with the random generated password.

KEY VARIABLES, DATA STRUCTURE AND CLASSES

Key variables:

Variable Name	Data type	Description
InputPassword	Char	This variable temporarily holds the Master Passwords user try to create.
MasterPasswordHash	Char	This variable holds the finalized Master Password meeting all Validation requirements
PasswordName	Char	This variable will hold temporarily hold the Name of the Passwords the user about to save
PasswordCategory	String	This variable holds the category temporarily
Password	Char	This variable holds the Password the user about to save temporarily
Authentication	Char	This variable holds the Master Password entered to access credentials.
ChangePassword	Char	This variable temporarily holds the password input in the password change text box.
StrengthScore	Integer	This variable will hold the score of the password strength calculation.

Data Structures:

- 1) **List / Array:** A list or array will be used to hold all whole character set for custom encryption development.
- 2) **SQL Database:** The database will be connected to the system to hold the encrypted passwords and hashed Master Password.
- 3) **Subroutine:** There will be several subroutines with different functionalities, and will be called in various pages as user interact with action buttons. These subroutines will be run internally with no user involvement.

TEST DATA (WHITE BOX TESTING)

1. Login page

Test data category	Input
Valid	Existing User ID and correct password under that User ID
Invalid	Existing User ID and wrong password under that User ID Wrong User ID and correct password Empty input field
Boundary case	Password or User ID exceeding limit (Max: 30 char)

2. Create account page

Test data category	Input
Valid	Non existing User ID and Master password
Invalid	Existing User ID or Master Password Empty input field
Boundary case	Password or User ID exceeding limit (Max: 30 char)

3. Master password hashing function

Test data category	Check
Valid	Master password hashed and stored in database table
Invalid	Master password hashed and not stored Master password is not hashed and stored

4. Password input page

Test data category	Input
Valid	Reference name and Category and Password
Invalid	Empty input field Special characters in User ID

Boundary case	Password or User ID exceeding limit (Max: 30 char)
----------------------	---

5. **Main page (with passwords categorized)**

Test data category	Input
Valid	Category match passwords under that category
Invalid	Empty category Category does not match passwords saved

6. **Password retrieval Authentication page**

Test data category	Input
Valid	Correct Master password
Invalid	Wrong Master password Empty input field
Boundary case	Password exceeding limit (Max: 30 char)

7. **Encryption and Decryption functions**

Test data category	Check
Valid	Correct encrypted password Correct decrypted password

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

Invalid	Wrong encrypted password Wrong decrypted password
Boundary case	Password exceeding limit (Max: 30 char)

8) **Change password function**

Test data category	Input
Valid	Any characters meeting the limit of max 30 char.
Invalid	Empty field
Boundary case	Password exceeding limit (Max: 30 char)

TEST DATA FOR ITERATIVE DEVELOPMENT (BLACK BOX)

No	Test purpose	Test input	Expected outcome	Success
1	Check if Log-in button navigates user to “Log-in” page	Clicking “Log-in” button	Navigate user to Login page	
2	Check if Login page functions as expected	Invalid: Input details of a non-existing account	Error message “No account under these details”	
	[Cover success criteria 1]	Valid: Input details of an existing account	Navigate user to the Main page	
3	Check if Create account button functions	Clicking “Create account” button	Navigate user to Create account page	
4	Check if Create account page functions as expected	Invalid: Input already existing account details	Error message “Already exists”	
	[Cover success criteria 3]	Valid: Input non-existing details	Navigate user to Log in page	
5	Create account Master password	Invalid: Test12	Error message “at least 8 characters needed”	

	Character length (At least 8 characters) [Cover success criteria 4]	Valid: Length.Test	Navigate to main page	
6	Uppercase letter check (validation rule 2) [Cover success criteria 4]	Invalid: length.test	Error message “at least 1 Uppercase letter needed”	
		Valid: Length.Test	Navigate to main page	
7	Special character check (validation rule 3) [Cover success criteria 4]	Invalid: Lengthtest	Error message “at least 1 special character”	
		Valid: Length@Test	Navigate to main page	
8	“Save” button runs the hashing function [Cover success criteria 5]	Clicking “Save” button	Hashed Master password in the database table and navigating user to main page	
9	Check if main page categories show all passwords under [Cover success criteria 16]	Clicking each category box	Each page must have its own category type passwords (eg: Medical-hospital, pharmacy)	
10	Dropdown list for category check [Cover success criteria 9]	Clicking the dropdown symbol	All 9 categories available for user to select one	
11	Check if reference name input box let user only input letters and numbers	Invalid: Barclays@134	Error message “No special characters are	
		Valid: Barclays Account	Allow user to save	

12	Check if the password input box inform the user about strength protocols [Cover success criteria 11]	Valid: Patches@belmont	Allow user to save the password.	
		Invalid: P@B	Message “Informing the user to enter a strong password due to short length”	
		Valid: @Oysteraccount	Allow user to save	
		Invalid: Oyster	Message “Informing the user to enter a strong password with use of special characters	
13	Check if “Auto password generator” button generates random passwords [Cover success criteria 12]	Clicking the button multiple times	Random password generated into the password input box	
14	Check if the “Save” button starts the Encryption [Cover success criteria 14]	Clicking the “Save” button.	The database table for password encryption must store the encryption version of the password	

15	Test if the “Cancel” button takes user to main page [Cover success criteria 15]	Clicking the “Cancel” button.	The main page should be displayed.	
16	Check if clicking a category on main page navigates the user to all passwords saved under that category [Cover success criteria 16]	Test 1: click Medical category	All passwords under Medical category only	
		Test 2: Click Financial category	All passwords under Financial category only	
		Test 3: Click Other category	All passwords that does not have specific category out of 9 categories listed	
17	Check if the “View” button navigates user to Authentication page [Cover success criteria 17]	Clicking the “view” button	Display the authentication page (where user needs to input Master password)	
18	Check if the Authentication input box grant access for actual user. [Master password: 912@HatchEnd] [Cover success criteria 19]	Valid: 912@HatchEnd	The user must be able to view the password he is being looking for.	
		Invalid: 912HatchEnd	Error message “Incorrect, please try again”	
19	Check if the Master password being compared with the	Input Master password and clicking “Enter” button	The function to compare the hashed master password begins	

	store master password			
20	Check if the “Back” button navigates user back to the main page	Click “Back” button	Main page must be displayed	
21	Check if the “Change” button functions	Clicking “Change” button	Password in the same location must be modified to the changed password	
22	Check if password strength checker function as expected	Valid: Patches@Belmont912	“Strong” with a green colour symbol	
	[Cover success criteria 22]	Invalid: P@B912	“Weak” with a red colour symbol	
23	Check if “back” button navigates user back to main page [Cover success criteria 23]	Clicking “back” button	Main page displayed	

Apart from the testing conducted as shown above in the table, a full functional testing (Black box testing) will be conducted as the final test. I would take multiple user (2-4 colleagues) and let them use the full software as they wish to check if the software meets the main requirements of saving password. Any changes will be made according to the upcoming errors detected or user dissatisfaction.

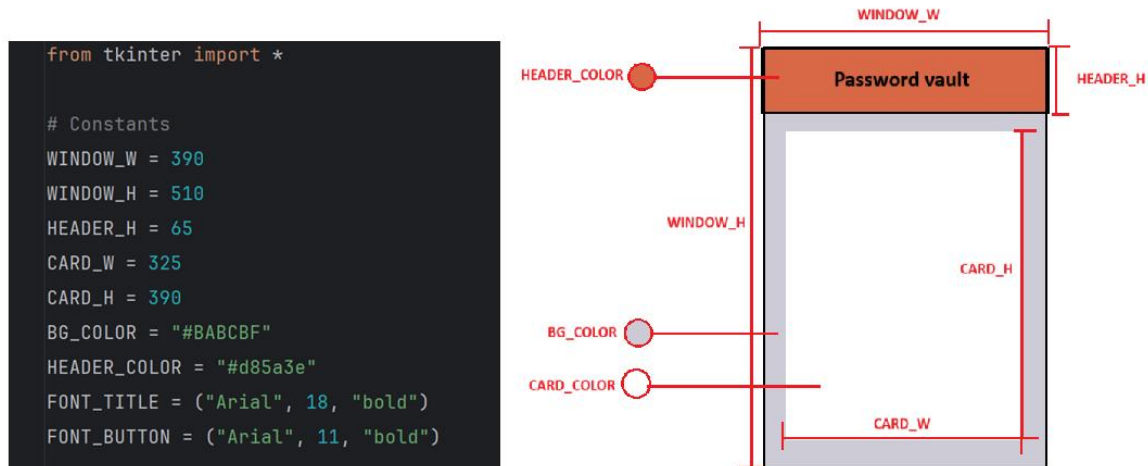
FUTHER DATA FOR POST-DEVELOPMENT PHASE

Development tools

- **Pycharm:** This is the most efficient software development tool to write my software as I will be using Python language and it is considered as all in one IDE. This IDE has strong debugging features (error detection) and has inbuilt GUI and Database in the Python library.
- **TKinter:** This is an inbuilt GUI feature in Pycharm. There will be no need to install GUI tools, because TKinter is available in the Pycharm library, meaning, just importing the TKinter library will allow me to design the software easily.
- **SQLite:** No need for extra installations as this feature comes inbuilt with Pycharm. SQLite is considered as a very beginner friendly and lightweight database. This is portable as the database can work in any computer just by copying the database file.

C. DEVELOPING THE CODED SOLUTION ("THE DEVELOPMENT STORY")

GUI Constants:

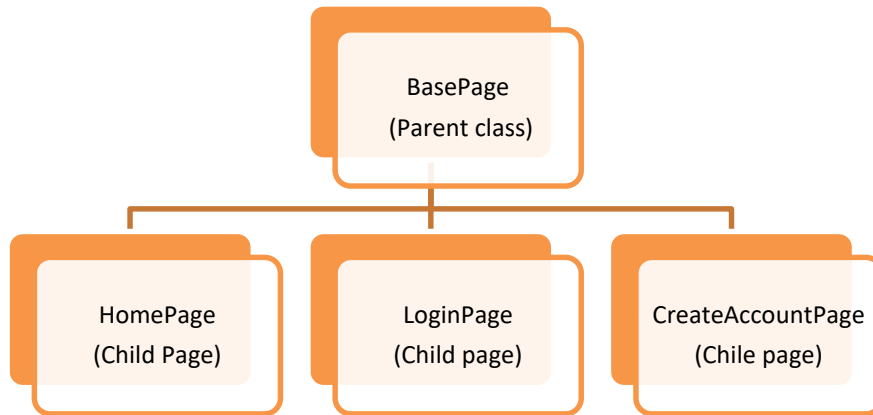


These constant variables were created to make sure I will not be assigning multiple variable names for the same values. The variables will be called throughout the program and this would make it easier for development since I can access the constant dimensions with their variable names.

Use of OOP (Object Oriented Programming):

I used Object Oriented Programming to make my program more organised and reusable. I created a parent class called **BasePage**, which contains all the common GUI components such as the window setup, header, background colour, and layout. Then I made child classes (all the pages where parent class will be used) that will **inherit** from the parent class.

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>



This allows each page to have its own unique elements while still keeping the shared design structure. Using inheritance reduces code repetition, make the program easier to maintain.

Parent class BasePage code:

```
class BasePage(Frame):
    def __init__(self, master):
        super().__init__(master)
        self.master = master
        self.master.resizable(False, False)
        self.master.geometry(f"{WINDOW_W}x{WINDOW_H}")
        self.master.configure(bg=BG_COLOR)

        # Header inside the page frame
        header = Frame(self, bg=HEADER_COLOR, width=WINDOW_W, height=HEADER_H)
        header.pack(side="top", fill="x")
        Label(header, text="Password Vault", bg=HEADER_COLOR, fg="black", font=FONT_TITLE).pack(pady=20)

        # Card
        self.card = Frame(self, bg="white", width=CARD_W, height=CARD_H)
        self.card.place(x=(WINDOW_W - CARD_W) // 2,
                        y=(WINDOW_H - HEADER_H - CARD_H) // 2 + HEADER_H)
        self.inner = Frame(self.card, bg="white")
        self.inner.place(relx=0.5, rely=0.5, anchor="center")

        self.pack(fill="both", expand=True)
```

master (parameter): The main TKinter window

Calls the constructor of the parent Frame and initialise the layout.

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

Output (RUN):

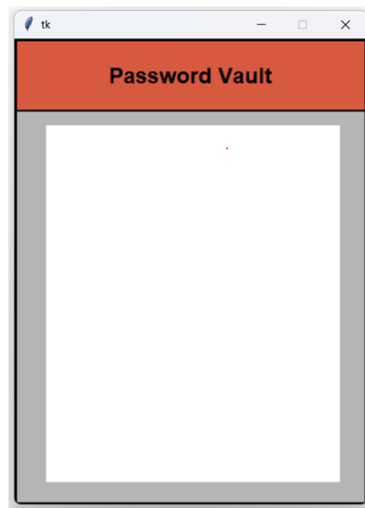


Figure 11: BasePage

DEVELOPMENT OF FRONT END

Page 1 [Homepage GUI]:

```
class HomePage(BasePage):
    def __init__(self, master):
        super().__init__(master)
        Button(self.inner, text="Login", font=("Arial", 14, "bold"), width=20,
               command=lambda: self.switch_page(LoginPage)).pack(padx=15, pady=10)
        Label(self.inner, text="or", font=("Arial", 12), bg="white").pack()
        Button(self.inner, text="Create Account", font=("Arial", 14, "bold"), width=20,
               command=lambda: self.switch_page(CreateAccountPage)).pack(padx=15, pady=10)

    2 usages
    def switch_page(self, page_class):
        self.destroy()
        page_class(self.master)
```

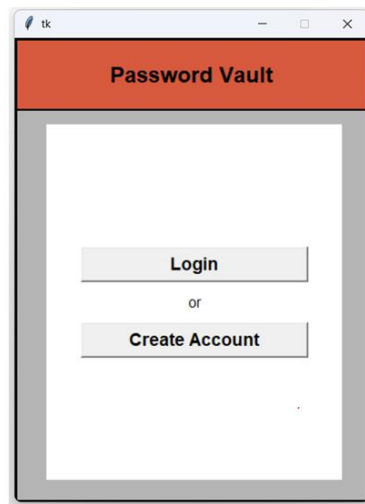
The two buttons for Login and Create account pages. Constant Variables are used.

switch_page in the function that will switch pages.

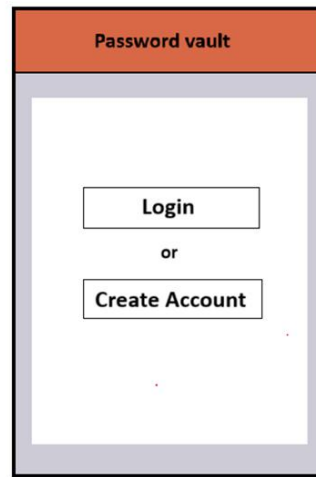
Self.destroy() remove the Homepage

Page_class opens the BasePage Parent class created previously

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>



Developed outcome



Design

Page 2 [Login page GUI]:

The username entry box where the user ID will be entered. Used on_click function to remove the default text when user click the input box

```
class LoginPage(BasePage):
    def __init__(self, master):
        super().__init__(master)
        Label(self.inner, text="Login", font=("Arial", 16, "bold"), bg="white").pack(pady=30)

        # Username
        username_entry = Entry(self.inner, width=25, font=("Arial", 13), bd=0.5, relief="solid")
        username_entry.pack(pady=25, ipady=8)
        username_entry.insert(index=0, string="Enter User ID")
        username_entry.config(fg="grey")

        def on_click(event):
            if username_entry.get() == "Enter User ID":
                username_entry.delete(first=0, last="end")

        username_entry.bind("<FocusIn>", on_click)

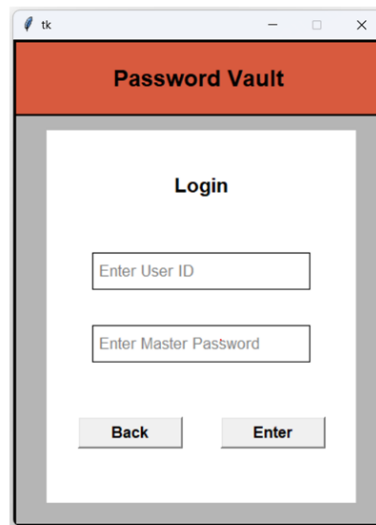
        # Password
        password_entry = Entry(self.inner, width=25, font=("Arial", 13), bd=0.5, relief="solid")
        password_entry.pack(pady=12, ipady=8)
        password_entry.insert(index=0, string="Enter Master Password")
        password_entry.config(fg="grey")

        def on_click(event):
            if password_entry.get() == "Enter Master Password":
                password_entry.delete(first=0, last="end")

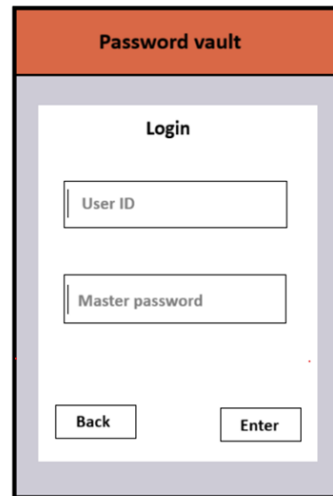
        password_entry.bind("<FocusIn>", on_click)
```

The password entry box where the Master Password will be entered to access the account.

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>



Development outcome



Design

Page 3 [Create Account page GUI]:

```
class CreateAccountPage(BasePage):
    def __init__(self, master):
        super().__init__(master)
        Label(self.inner, text="Create Account", font=("Arial", 16, "bold"), bg="white").pack(pady=20)
        # Username
        username_entry = Entry(self.inner, width=25, font=("Arial", 12), bd=1, relief="solid")
        username_entry.pack(pady=10, ipady=6)
        username_entry.insert(index=0, string=" Enter User ID here")
        username_entry.config(fg="grey")

        def on_click(event):
            if username_entry.get() == " Enter User ID here":
                username_entry.delete(first=0, last="end")

        username_entry.bind("<FocusIn>", on_click)
```

The User ID entry box where the user will create a unique user ID. This code contains only the GUI, later, the User ID comparison code will be added to check if User ID already exists.

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

```
# Password
password_entry = Entry(self.inner, width=25, font=("Arial", 12), bd=1, relief="solid")
password_entry.pack(pady=20, ipady=6)
password_entry.insert(index: 0, string: " Enter Master Password here")
password_entry.config(fg="grey")

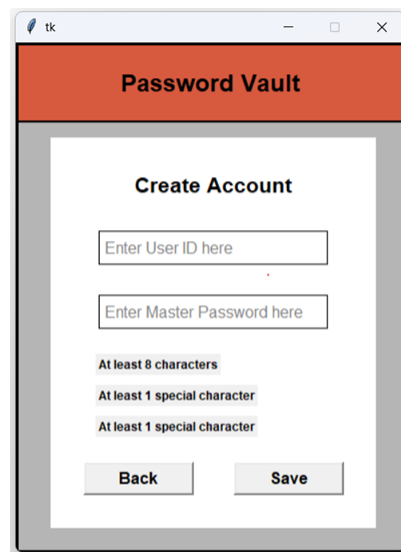
def on_click(event):
    if password_entry.get() == " Enter Master Password here":
        password_entry.delete( first: 0, last: "end")

password_entry.bind("<FocusIn>", on_click)

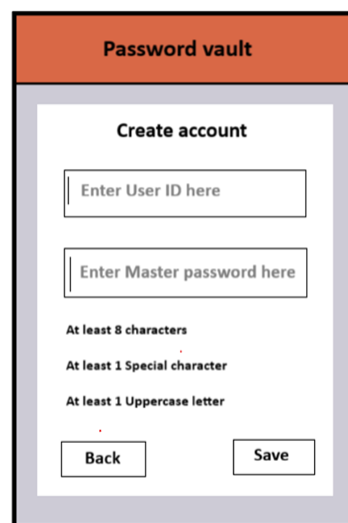
# Validation rules Label
Label(self.inner, text="At least 8 characters", font=("Arial", 9, "bold")).pack(anchor="w", pady=5)
Label(self.inner, text="At least 1 special character", font=("Arial", 9, "bold")).pack(anchor="w", pady=5)
Label(self.inner, text="At least 1 special character", font=("Arial", 9, "bold")).pack(anchor="w", pady=5)
```

The password entry box where the Master Password will be created under the Validation rules

The Labels containing the Validation rules for the user to understand the creation requirements. No validation logic in this code.



Developed outcome



Design

Page 4 [Main page GUI]:

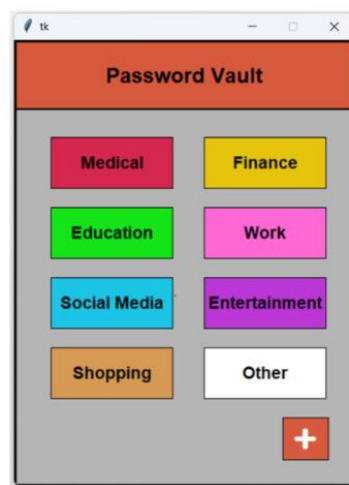
```
class Mainpage(BasePage):
    def __init__(self, master):
        super().__init__(master)
        self.card.configure(bg="#B5B5B5")
        self.inner.configure(bg="#B5B5B5")
```

Calling of the Parent class (Basepage) and modifying the card background to fit the design.

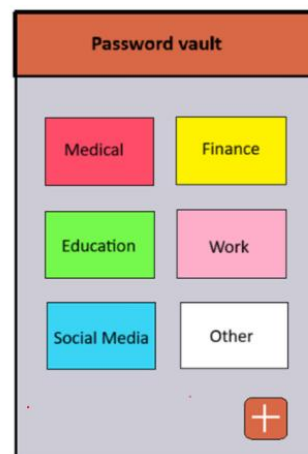
#Buttons

```
Medicalbtn= Button(self.inner, text="Medical", bg="#D6274F", fg="black", font=("Arial", 14, "bold"), width=11, height=2, command=lambda: self.store_password("Medical"))
Medicalbtn.place(x=5, y=15)
Financebtn = Button(self.inner, text="Finance", bg="#E6C40D", fg="black", font=("Arial", 14, "bold"), width=11, height=2, command=lambda: self.store_password("Finance"))
Financebtn.place(x=180, y=15)
Educationbtn = Button(self.inner, text="Education", bg="#15E518", fg="black", font=("Arial", 14, "bold"), width=11, height=2, command=lambda: self.store_password("Education"))
Educationbtn.place(x=5, y=95)
Workbtn = Button(self.inner, text="Work", bg="#FF69D5", fg="black", font=("Arial", 14, "bold"), width=11, height=2, command=lambda: self.store_password("Work"))
Workbtn.place(x=180, y=95)
SocialMediabtn = Button(self.inner, text="Social Media", bg="#1CC4E5", fg="black", font=("Arial", 14, "bold"), width=11, height=2, command=lambda: self.store_password("Social Media"))
SocialMediabtn.place(x=5, y=175)
Entertainmentbtn = Button(self.inner, text="Entertainment", bg="#BC38D6", fg="black", font=("Arial", 14, "bold"), width=11, height=2, command=lambda: self.store_password("Entertainment"))
Entertainmentbtn.place(x=180, y=175)
Shoppingbtn = Button(self.inner, text="Shopping", bg="#D69A54", fg="black", font=("Arial", 14, "bold"), width=11, height=2, command=lambda: self.store_password("Shopping"))
Shoppingbtn.place(x=5, y=255)
Otherbtn = Button(self.inner, text="Other", bg="white", fg="black", font=("Arial", 14, "bold"), width=11, height=2, command=lambda: self.store_password("Other"))
Otherbtn.place(x=180, y=255)
Plusbtn= Button(self.inner, text="+", bg="#d85a3e", fg="white", font=("Arial", 18, "bold"), command=lambda: self.store_password("+"))
Plusbtn.place(x=270, y=335)
```

All the category buttons including the Plus symbol button for user store passwords in the system under their account.



Development outcome



Design

Page 5 [Store Password page GUI]

The dropdown input box where the user can pick a related category for the password

```
class StorePasswordpage(BasePage):
    def __init__(self, master):
        super().__init__(master)

        #dropdown input
        options = ["Medical", "Finance", "Education", "Work", "Social Media", "Entertainment", "Shopping", "Other"]
        dropdown = ttk.Combobox(self.inner, values=options, width=23, font=("Arial", 13))
        dropdown.set("Category")
        dropdown.pack(pady=15, ipady=6)

        #Reference input
        Refname_entry = Entry(self.inner, width=25, font=("Arial", 12), bd=1, relief="solid")
        Refname_entry.pack(pady=15, ipady=6)
        Refname_entry.insert(index=0, string=" Reference name")
        Refname_entry.config(fg="grey")

        def on_click(event):
            if Refname_entry.get() == " Reference name":
                Refname_entry.delete(first=0, last="end")

        Refname_entry.bind("<FocusIn>", on_click)

        #Store Password input
        SPASSWORD_entry = Entry(self.inner, width=25, font=("Arial", 12), bd=1, relief="solid")
        SPASSWORD_entry.pack(pady=15, ipady=6)
        SPASSWORD_entry.insert(index=0, string=" Password")
        SPASSWORD_entry.config(fg="grey")

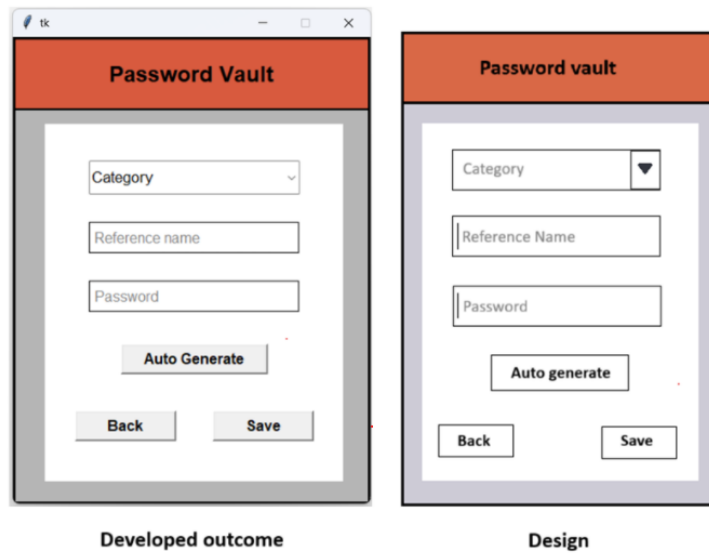
        def on_click(event):
            if SPASSWORD_entry.get() == " Password":
                SPASSWORD_entry.delete(first=0, last="end")

        SPASSWORD_entry.bind("<FocusIn>", on_click)

        #Buttons
        AutoGenbtn=Button(self.inner, text="Auto Generate", font=FONT_BUTTON, width=15)
        AutoGenbtn.pack(pady=20)
        Button(self.inner, text="Save", font=FONT_BUTTON, width=10, command=lambda: print("Account Created:")).pack(side=
        Button(self.inner, text="Back", font=FONT_BUTTON, width=10, command=lambda: self.switch_page(Mainpage)).pack(side=
```

The Auto Password Generate button GUI where users can click to pop up a random password on the password entry box.

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>



Page 6 [Show Page GUI]

```
class Show_page(BasePage):
    def __init__(self, master):
        super().__init__(master)
        name_label = Label(self, text="Name: ", font=("Arial", 13))
        name_label.place(x=60, y=120)
        Password_label = Label(self, text="Password: ", font=("Arial", 13))
        Password_label.place(x=60, y=170)

        #Change input box
        Change_entry = Entry(self.inner, width=22, font=("Arial", 14), bd=1, relief="solid")
        Change_entry.place(x=40, y=140)
        Change_entry.insert(index=0, string=" Change Password here")
        Change_entry.config(fg="grey")

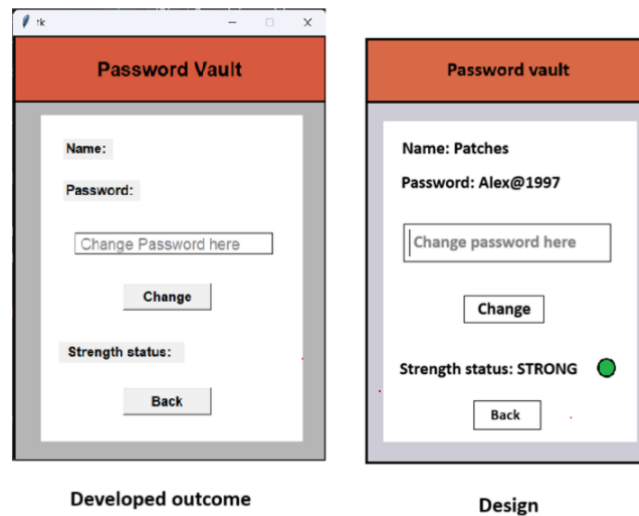
        def on_click(event):
            if Change_entry.get() == " Change Password here":
                Change_entry.delete(first=0, last="end")

        Change_entry.bind("<FocusIn>", on_click)
        Button(self.inner, text="Change", font=("Arial", 13), width=10).place(x=100, y=200)

        Strength_status_label = Label(self.inner, text="Strength status: ", width=15, font=("Arial", 13))
        Strength_status_label.place(x=20, y=270)

        Button(self.inner, text="Back", font=("Arial", 13), width=10).place(x=100, y=325)
```

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>



FRONT-END TEST TABLE:

Page	Test item	Purpose	Success
Page 1 [Home Page]	Login button	Check if clicking the button navigates user to Login page	☒
	Create Account button	Check if clicking the button navigates user to Create Account page	☒
Page 2 [Login Page]	User ID entry box	Check if the entry box allows user to enter characters	☒
	Password entry box	Check if the entry box allows user to enter characters	☒
	Enter button	Check if user will be taken to the Main page	☒
	Back button	Check if the user will be taken to the previous page (Home Page)	☒
Page 3 [Create Account Page]	User ID entry box	Check if the entry box allows user to enter characters	☒
	Password entry box	Check if the entry box allows user to enter characters	☒
	Save button	Check if user will be taken to the Main page	☒
	Back button	Check if the user will be taken to the previous page (Home Page)	☒

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

Page 4 [Main Page]	All category buttons	Check if all the category buttons click successfully	☒
	Plus symbol button	Check if the user will be taken to the Store Password page	☒
Page 5 [Store Password Page]	Category dropdown	Check if all the categories functions in the dropdown list	☒
	Reference name entry box	Check if user can enter any character to the entry box	☒
	Password entry box	Check if user can enter any character to the entry box	☒
	Auto generate button	Check if the button functions	☒
	Save button	Check if the save Button functions	☒
	Back button	Check if the user will be taken to the Main page	☒

Reference:

1. **YouTube video:** (NeuralNine, Tkinter Beginner Course - Python GUI Development, 2021) **Date (18/09/2025)**
2. **YouTube video:** (BroCode, 2020) **Date (18/09/2025)**
3. **Website:** (Fitzpatrick, 2014) **Date (18/09/2025)**
4. **Website:** (Amos, 2024) **Date (18/09/2025)**
5. **YouTube:** (Codemy.com, 2020) **Date (18/09/2025)**
6. **Website:** (Roseman, 2025) **Date (18/09/2025)**

DEVELOPMENT OF BACK END

I. Password strength checker

17/11/2025

Prototype 1

```

1 import string
2 def strength_checker():
3     password = "0c1@"
4     length = len(password)
5     Lscore = 0
6     Cscore = 0
7     cU = 0
8     cL = 0
9     cD = 0
10    cS = 0
11
12    # Length points
13    if length >= 10:
14        Lscore += 3
15    elif (length < 10) and (length > 5):
16        Lscore += 2
17    elif (length <= 5) and (length > 1):
18        Lscore += 1
19
20    upper = string.ascii_uppercase
21    lower = string.ascii_lowercase
22    digits = string.digits
23    symbols = "!@#%&*()-_+[]{};:.,<>?"
24
25    # Uppercase check
26    for i in range(length):
27        for j in range(len(upper)):
28            if password[i] == upper[j]:
29                cU += 1
30
31    # Lowercase check
32    for i in range(length):
33        for j in range(len(lower)):
34            if password[i] == lower[j]:
35                cL += 1
36
37    # Digit check
38    for i in range(length):
39        for j in range(len(digits)):
40            if password[i] == digits[j]:
41                cD += 1
42
43    # Symbol check
44    for i in range(length):
45        for j in range(len(symbols)):
46            if password[i] == symbols[j]:
47                cS += 1
48
49    # Character Score logic
50    if cU > 1 and cL > 1 and cD > 1 and cS > 1:
51        Cscore += 3
52    elif cU == 1 and cL == 1 and cD == 1 and cS == 1:
53        Cscore += 2
54    elif cU < 1 or cL < 1 or cD < 1 or cS < 1:
55        Cscore += 1
56    else:
57        Cscore += 0
58
59    total_score = Cscore + Lscore
60
61    if total_score <= 2:
62        print("weak")
63    elif 3 <= total_score <= 4:
64        print("medium strong")
65    elif 5 <= total_score <= 6:
66        print("Strong")
67    else:
68        print("ERROR")

```

Nested for loops used to count the number of same type characters (numbers)

Score for the length of the password user saving. If password too short score will be lower

Prototype analysis: The logic of this prototype is what I discussed in the Analysis stage where the length and the character score are each rate between 1-3 and get the sum to finalise the strength as below:

If **sum**: less than or equal to 2: **“Weak”**

3 or 4: **“Medium strong”**

5 or 6: **“Strong”**

Black box testing 1:

No	Test Password	Expected outcome	Actual outcome	Success
1	PeterHend@2004	Strong	Medium strong	Fail
2	AL#23	Weak	Weak	Pass
3	AL#@23	Weak	Medium strong	Fail
4	OCRcomputing@2025	Strong	Medium strong	Fail
5	OCRcomputing@#2025	Strong	Strong	Pass
6	DHL.package25	Medium strong	Medium strong	Pass
7	Dustin	Weak	Medium strong	Fail
8	OCR.123	Weak	Medium strong	Fail

Most of the passwords tested are **failed** according to the above test table. Leading to a conclusion of this prototype logic must be changed.

One symbol changes the status from current status to a Stronger status. This is not valid as AL#@23 should also be “Weak”

Evidence:

```

4  def strength_checker():
5      password = "AL#@23"
6      length = len(password)
7      Lscore = 0

```

Test prototype x

strength_checker()

"C:\Users\Anodhya Lankathilaka\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\medium strong"

Clearly, the password “AL#@23” is not a medium strong password. Removing the symbol “@” will outcome the result as Weak. Hence, this prototype is not a valid version

```

4  def strength_checker():
5      password = "OCRcomputing@2025"
6      length = len(password)
7      Lscore = 0

```

Test prototype x

strength_checker()

"C:\Users\Anodhya Lankathilaka\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\medium strong"

The password “OCRcomputing@2025” is a strong password. The prototype outputs medium strong resulting being a failed test.

```

4  def strength_checker():
5      password = "Dustin"
6      length = len(password)
7      Lscore = 0

```

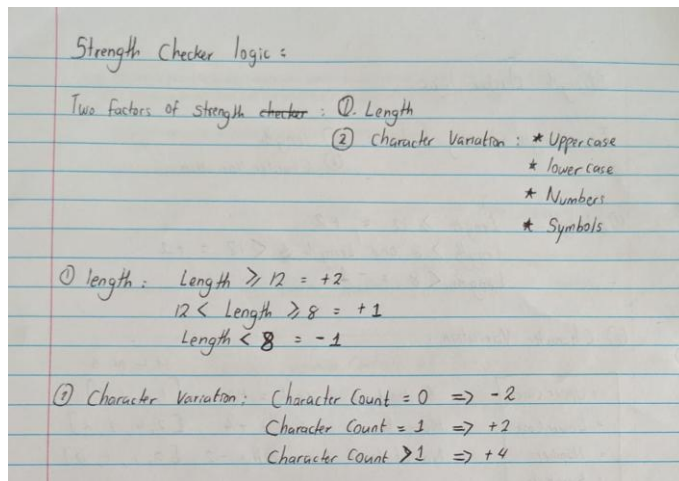
Test prototype x

strength_checker()

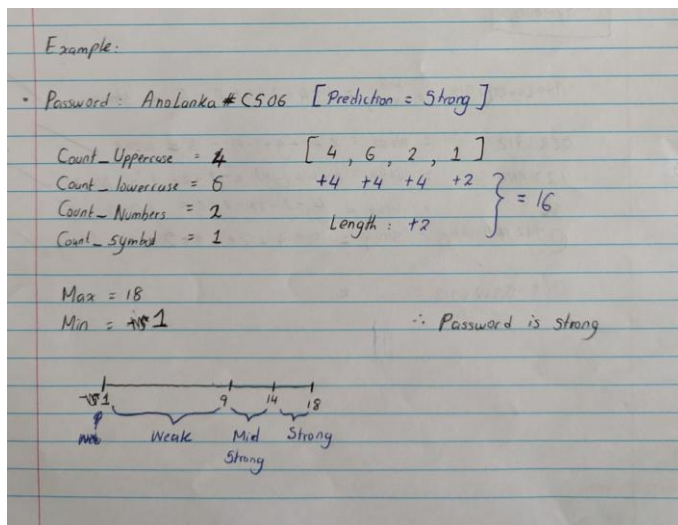
"C:\Users\Anodhya Lankathilaka\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\medium strong"

Clearly, the password “Dustin” is a Weak password as it refers to a name and could take minutes for a computer to crack. The prototype version does not provide a valid result.

22/11/2025

Prototype 2

The logic used in the first prototype is modified as shown in the image. The length factor is modified to increase and decrease the score according to the length of the password. And the character variation score system increment and decrement the score depending on the password characters used, rather than only incrementing as in prototype 1. This will reduce score points for each character type if user haven't used.



Example use: The password shown is expected to outcome password status as “Strong”. The counts of each character types are taken, and according to the number of counts, the score is changed. As shown in the image the character variation score will be 14 and with length score the final score will be 16. 16 is between 14 and 18 concluding the password as “Strong”

Justification:

In prototype 1, the logic used failed to check several passwords (passwords that didn't met certain conditions). However, the new prototype logic design shown above improves the strength checker by introducing a new scoring system, where passwords will get incremented for strong factors user have met and also decrements if individuals haven't met certain factors. This logic successfully passes all test passwords that failed in prototype 1. Hence, prototype 2 will be the final prototype of Password Strength checker function.

```

1- def strength_checker():
2-     password = "Dustin"
3-     length = len(password)
4-     score = 0
5-     cU = 0 #Uppercase count
6-     cL = 0 #Lowercase count
7-     cD = 0 #Numbers count
8-     cS = 0 #Symbol count
9-
10-    #Length score
11-    if length >= 12:
12-        score = score + 2
13-    elif 8 <= length < 12:
14-        score = score + 1
15-    else:
16-        score = score - 1
17-
18-    upper = string.ascii_uppercase
19-    lower = string.ascii_lowercase
20-    digits = string.digits
21-    symbols = "!@#$%^&*()-_+[]{};:,<.>?"
22-
23-    # Uppercase count
24-    for i in range(length):
25-        for j in range(len(upper)):
26-            if password[i] == upper[j]:
27-                cU += 1
28-
29-    # Lowercase count
30-    for i in range(length):
31-        for j in range(len(lower)):
32-            if password[i] == lower[j]:
33-                cL += 1
34-
35-    # Digit count
36-    for i in range(length):
37-        for j in range(len(digits)):
38-            if password[i] == digits[j]:
39-                cD += 1
40-
41-    # Symbol count
42-    for i in range(length):
43-        for j in range(len(symbols)):
44-            if password[i] == symbols[j]:
45-                cS += 1
46-
47-    #Character variation score
48-    character = [cU, cL, cD, cS]
49-
50-    for i in range(0, len(character)):
51-        current = character[i]
52-        if current == 0:
53-            score = score - 2
54-        elif current == 1:
55-            score = score + 2
56-        elif current > 1:
57-            score = score + 4
58-
59-    # status
60-    if score >= 14:
61-        print("Strong")
62-    elif 9 < score <= 13:
63-        print("Medium strong")
64-    else:
65-        print("Weak")
66-
67-
68-

```

The length score system is changed. The score variable is left as one variable to reduce the usage of multiple variables (used in prototype 1)

The character variation uses an array with character counts as its elements. This array is used to loop around each character count and score according to the if else conditions

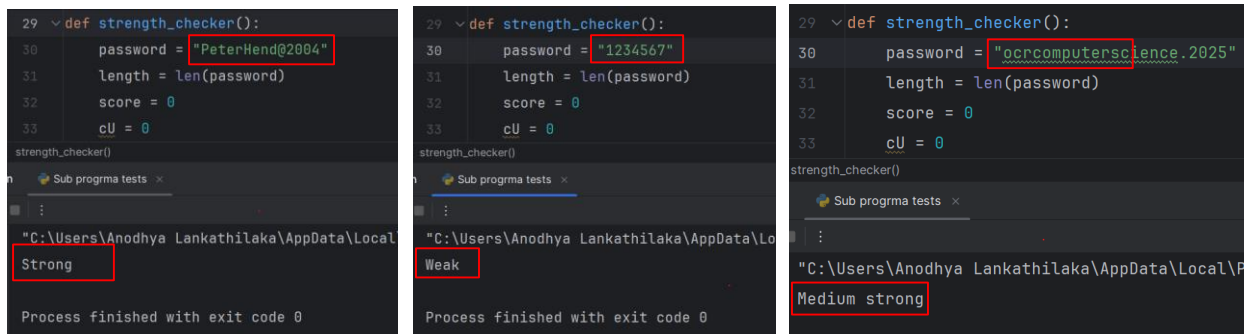
Black box testing 2

No	Test Password	Expected outcome	Actual outcome	Success
1	PeterHend@2004	Strong	Strong	pass
2	Anodhya@y13	Medium strong	Medium strong	pass
3	AL#@23	Weak	Weak	pass
4	OCRcomputing@2025	Strong	Strong	pass
5	1234567	Weak	Weak	pass
6	@#!Robin6	Medium strong	Medium strong	pass
7	ocrcomputerscience.2025	Medium strong	Medium strong	pass
8	Cs@3	Weak	Weak	pass

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

Evidence:

Below are three evidences to prove the outcome of the new password strength checker prototype.



Reference:

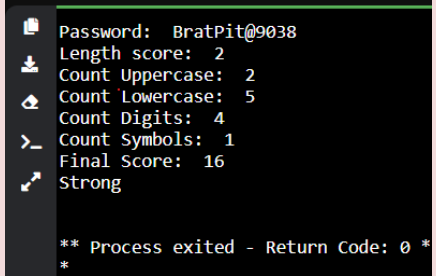
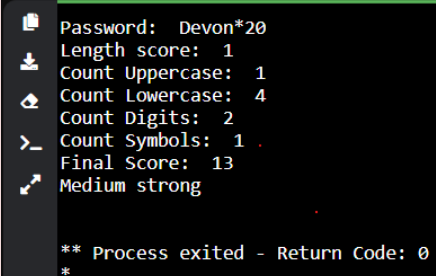
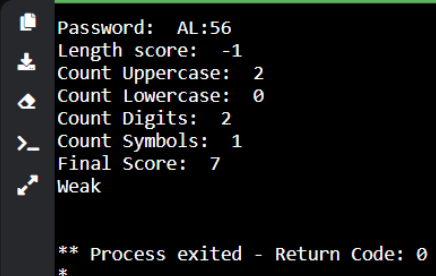
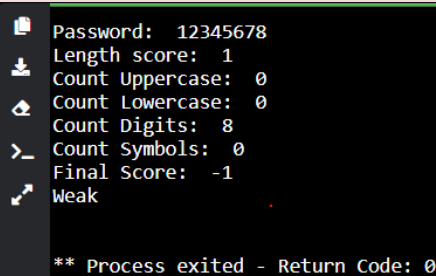
- 1) **Python library:** (Pyhton3.x.documrntation, 2024) **Date:** 22/11/2025
- 2) **YouTube video:** (NeuralNine, 2022) **Date:** 22/11/2025
- 3) **Password Strength checker:** (NordPass, 2025) **Date:** 22/11/2025

Review:

In this module where I created the function for password strength checker, I provided two prototypes. Prototype 1 did not produce the expected results concluding it as a failed prototype. However, Prototype 2 where I rebuild a new logic for score system and introduced a new score-strength scale, the passwords that failed in Prototype 1 were successfully resulting the expected outcomes. In conclusion, I decided to leave prototype 2 as the final prototype as it met all the tests successfully and the code is logically efficient.

White box testing will be presented in a table as shown below.

White box testing of Strength Checker:

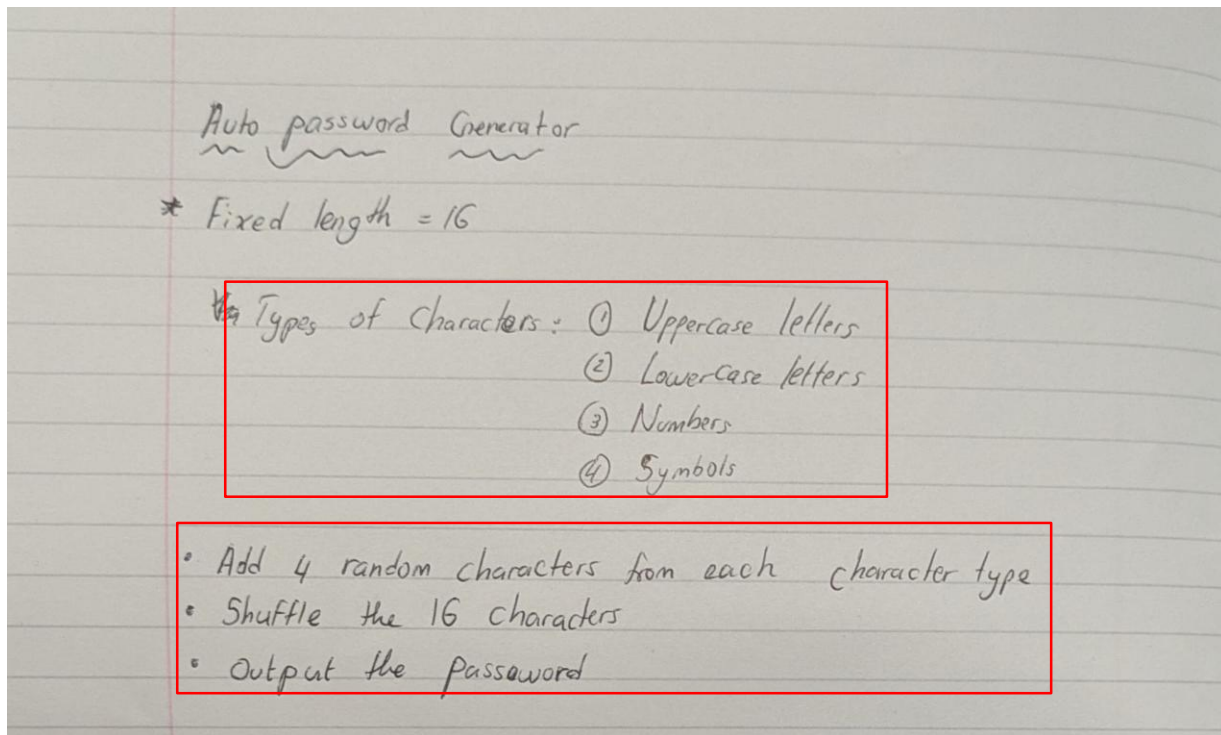
No	Test input	Expected outcome		Actual outcome	Evidence	Success
1	BratPit@9038	Length score	+2	+2	 <pre> Password: BratPit@9038 Length score: 2 Count Uppercase: 2 Count Lowercase: 5 Count Digits: 4 Count Symbols: 1 Final Score: 16 Strong ** Process exited - Return Code: 0 * </pre>	PASS
		Count Uppercase	2	2		
		Count Lowercase	5	5		
		Count Digits	4	4		
		Count Symbol	1	1		
		Final Score	16	16		
		Strength status	Strong	Strong		
2	Devon*20	Length score	+1	+1	 <pre> Password: Devon*20 Length score: 1 Count Uppercase: 1 Count Lowercase: 4 Count Digits: 2 Count Symbols: 1 Final Score: 13 Medium strong ** Process exited - Return Code: 0 * </pre>	PASS
		Count Uppercase	1	1		
		Count Lowercase	4	4		
		Count Digits	2	2		
		Count Symbol	1	1		
		Final Score	13	13		
		Strength status	Medium	Medium		
3	AL:56	Length score	-1	-1	 <pre> Password: AL:56 Length score: -1 Count Uppercase: 2 Count Lowercase: 0 Count Digits: 2 Count Symbols: 1 Final Score: 7 Weak ** Process exited - Return Code: 0 * </pre>	PASS
		Count Uppercase	2	2		
		Count Lowercase	0	0		
		Count Digits	2	2		
		Count Symbol	1	1		
		Final Score	7	7		
		Strength status	Weak	Weak		
4	12345678	Length score	+1	+1	 <pre> Password: 12345678 Length score: 1 Count Uppercase: 0 Count Lowercase: 0 Count Digits: 8 Count Symbols: 0 Final Score: -1 Weak ** Process exited - Return Code: 0 * </pre>	PASS
		Count Uppercase	0	0		
		Count Lowercase	0	0		
		Count Digits	8	8		
		Count Symbol	0	0		
		Final Score	-1	-1		
		Strength status	Weak	Weak		

28/11/2025

2. Auto password generator

Prototype 1:

Password generator logic: as discussed in Analysis stage



All passwords will have a fixed length of 16 characters to maintain secure passwords. The **Random** library will be used throughout the code of generating a password. Once four random characters from each character type is gathered to a one array, the elements will be shuffled and the elements will be pushed into one variable as the final password. This will be appeared on the password entry box as user clicks “Auto Generate” button.

Solution code:

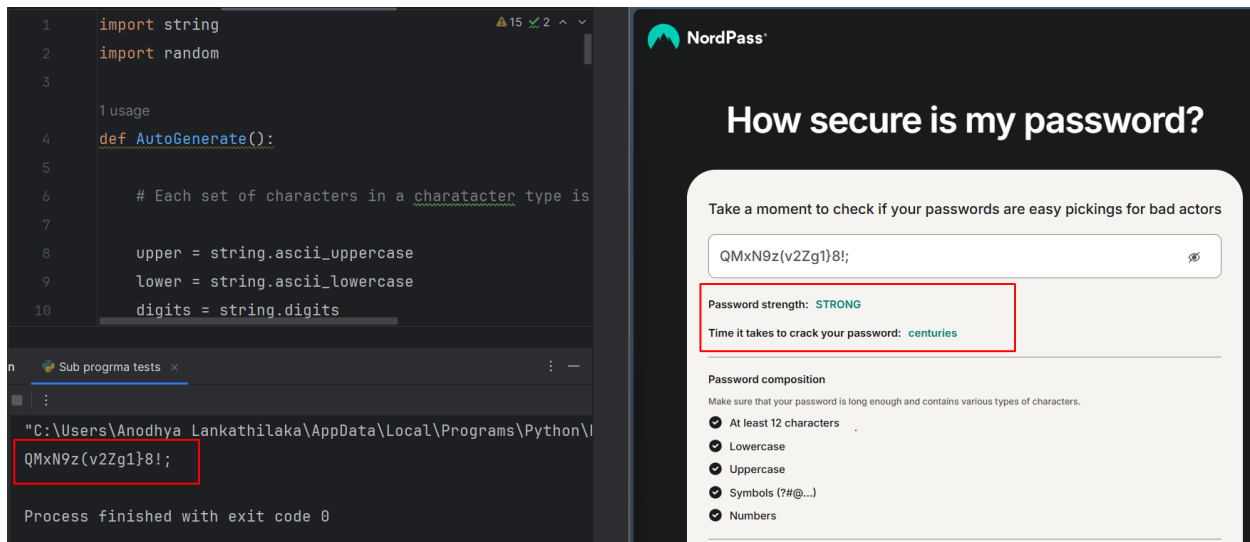
```
1 import string
2 import random
3
4 def AutoGenerate():
5
6     # Each set of characters in a character type is assigned a variable
7
8     upper = string.ascii_uppercase
9     lower = string.ascii_lowercase
10    digits = string.digits
11    symbols = "!@#$%^&*()-_+[]{};:,<.>?"
12
13    character_type = [upper, lower, digits, symbols]
14
15    # Creation of password with 4 characters in each character type
16    password_chars = []
17
18    for i in range(0, 4):
19        for j in range(0, 4):
20            password_chars = password_chars + [random.choice(character_type[i])]
21
22    # Shuffle code for password_chars
23    P_length = len(password_chars) # Length of password chars
24    for i in range(P_length):
25        j = random.randint(0, P_length-1)
26        hold = password_chars[j]
27        password_chars[j] = password_chars[i]
28        password_chars[i] = hold
29
30    # Combine the password characters to a string
31
32    password = ""
33    for i in range(0, len(password_chars)):
34        password = password + password_chars[i]
35    return password
36
37 print(AutoGenerate())
```

Nested FOR loops used to pick 4 characters randomly for each character type.

Each element of the array being shuffled with a random element in the array. This loop I used has the same swap logic as I learned in sort algorithms.

Black box testing:

I tested the strength of 5 different passwords from an actual password strength checker and all the passwords received a status of “**Strong**” with time to crack as centuries. Hence this code provides secure passwords



Prototype analysis: This prototype has various inefficiencies even though it produces strong password for every output.

- 1) **Time complexity:** The run time slows down due to **password_char** which is under creation of password with 4 characters in each character type. A new list will be created each time it passes through the FOR loop. Therefore, the solution is to use append to push characters into the array which is more convenient.
- 2) **Shuffle algorithm:** The shuffle algorithm is biased. After researches about shuffle algorithm, I found that “Fisher Yates” shuffle algorithm is an unbiased and efficient shuffle algorithm. Therefore, I will be developing Fisher Yates algorithm in my next prototype.

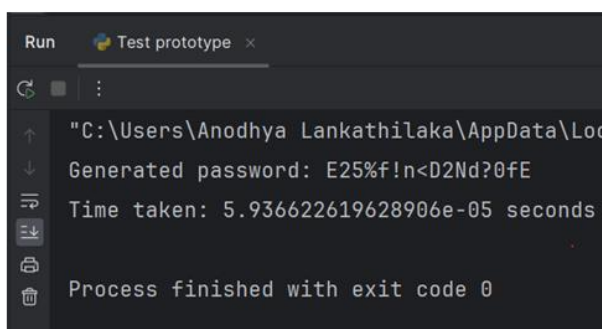
Prototype 2

```
1 # Creation of password with 4 characters in each character type
2 password_chars = []
3
4 for i in range(0, 4):
5     for j in range(0, 4):
6         password_chars.append(random.choice(character_type[i]))
7
8 # Shuffle code for password chars
9 shuffled = []
10 while len(password_chars) > 0:
11     rand_index = random.randrange(0, len(password_chars))
12     shuffled.append(password_chars[rand_index])
13     password_chars.pop(rand_index)
14 password_chars = shuffled
15
```

The shuffle algorithm is recreated as the Fisher Yates shuffle algorithm which is more efficient compared to prototype 1 shuffle algorithm. This outputs more random shuffled passwords.

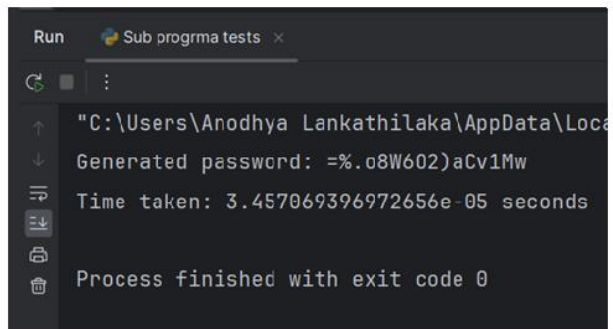
Test Evidence:

By using **import time**, I found the execution times of both prototypes. The second prototype is 25 microseconds faster than prototype 1. Prototype 1 showed an execution time of 59 microseconds while Prototype 2 showed an execution time of 34 microseconds. This proves that the second prototype is more efficient compared to prototype 1.



```
Run Test prototype x
"C:\Users\Anodhya Lankathilaka\AppData\Local\Temp\..."
Generated password: E25%fn<D2Nd?0fE
Time taken: 5.936622619628906e-05 seconds
Process finished with exit code 0
```

Prototype 1 execution time



```
Run Sub progrma tests x
"C:\Users\Anodhya Lankathilaka\AppData\Local\Temp\..."
Generated password: =%.o8W602)aCv1Mw
Time taken: 3.457069396972656e-05 seconds
Process finished with exit code 0
```

Prototype 2 execution time

References:

- 1) **YouTube Video:** (channel, 2020) **Date:** 29/11/2025
- 2) **YouTube Video:** (TechWithTim, 2023) **Date:** 29/11/2025
- 3) **Password Strength checker:** (NordPass, 2025) **Date:** 22/11/2025

Review:

In this module where I created an Auto Password generator, I came up with two prototypes. Prototype 1 was a basic password strength checker I build with no resources such as “Fisher Yate algorithm” or append technique for “password_char” array. However, after code optimization by using Fisher Yate shuffle algorithm and append technique, the execution time for Prototype 2 doped significantly with strong password as output proving it to be the final prototype of this module.

White box testing:

```
# Creation of password with 4 characters in each character type
password_chars = []

for i in range(0, 4):
    for j in range(0, 4):
        password_chars.append(random.choice(character_type[i]))
print(password_chars)
```

```
24 # Shuffle code for password_chars
25
26 shuffled = []
27 while len(password_chars) > 0:
28     rand_index = random.randrange(0, len(password_chars))
29     shuffled.append(password_chars[rand_index])
30     print(shuffled)
31     password_chars.pop(rand_index)
32 password_chars = shuffled
33
```

```
34 # Combine the password characters to a string
35
36 password = ""
37 for i in range(0, len(password_chars)):
38     password = password + password_chars[i]
39     print(password)
40 return password
```

4 random character types in each loop

```
[ 'Y', 'A', 'Z', 'J' ]
[ 'Y', 'A', 'Z', 'J', 'z', 'f', 't', 'b' ]
> [ 'Y', 'A', 'Z', 'J', 'z', 'f', 't', 'b', '3', '7', '2', '4' ]
[ 'Y', 'A', 'Z', 'J', 'z', 'f', 't', 'b', '3', '7', '2', '4', 'i', '#', '+', ']' ]
```

```
> [ '3' ]
[ '3', 'A', 't' ]
[ '3', 'A', 't', 'f' ]
[ '3', 'A', 't', 'f', '#' ]
[ '3', 'A', 't', 'f', '#', 'b' ]
[ '3', 'A', 't', 'f', '#', 'b', '7' ]
[ '3', 'A', 't', 'f', '#', 'b', '7', 'z' ]
[ '3', 'A', 't', 'f', '#', 'b', '7', 'z', '2' ]
[ '3', 'A', 't', 'f', '#', 'b', '7', 'z', '2', '+' ]
[ '3', 'A', 't', 'f', '#', 'b', '7', 'z', '2', '4' ]
[ '3', 'A', 't', 'f', '#', 'b', '7', 'z', '2', '4', 'i' ]
[ '3', 'A', 't', 'f', '#', 'b', '7', 'z', '2', '4', 'i', ']' ]
[ '3', 'A', 't', 'f', '#', 'b', '7', 'z', '2', '4', 'i', ']', '3' ]
[ '3', 'A', 't', 'f', '#', 'b', '7', 'z', '2', '4', 'i', ']', '3', '4' ]
[ '3', 'A', 't', 'f', '#', 'b', '7', 'z', '2', '4', 'i', ']', '3', '4', 'Y' ]
[ '3', 'A', 't', 'f', '#', 'b', '7', 'z', '2', '4', 'i', ']', '3', '4', 'Y', '2' ]
```

```
3
3A
3At
3Atf
3Atf#
3Atf#b
3Atf#b7
3Atf#b7;
3Atf#b7;Z
3Atf#b7;Z+
3Atf#b7;Z+;
3Atf#b7;Z+;Z
3Atf#b7;Z+;Z3
3Atf#b7;Z+;Z34
3Atf#b7;Z+;Z34Y
3Atf#b7;Z+;Z34Y2
3Atf#b7;Z+;Z34Y2
```

Password: 3Atf#b7;Z+]Zj4y2

30/11/2025

3. Encryption & Decryption

I decided to program Encryption and Decryption at the same time, so that I can check if the same logic for encryption can be reversed to logically move backwards to get the actual password. I will be building the same encryption I have discussed in the analysis (Caesar cipher) and I will search ways to improve the code and logic if needed.

Prototype 1

Encryption:

```
1 import random
2 import string
3
4 password = "ElonMusk@SpaceX"
5 print("Password = ",password)
6 def encryption(password):
7     symbols = "!@#$%^&*()-_+[]{};:,.<>?"
8     # every character a user can input
9     Keyboard = string.ascii_uppercase + string.ascii_lowercase + string.digits + symbols
10
11
12     shift_key = 9
13
14     # creating a new shifted keyboard variable
15     shifted = Keyboard[shift_key:] + Keyboard[:shift_key]
16
17     # Encryptes the password
18     encrypted = []
19     for char in password:
20         index = Keyboard.find(char)
21         encrypted.append(shifted[index])
22
23     # Join the encrypted array to a one string
24     Encrypted_password = ""
25     for i in range(0, len(encrypted)):
26         Encrypted_password = Encrypted_password + encrypted[i]
27     return Encrypted_password
28
29 encrypted_pw = encryption(password)
30 print("Encrypted = ",encrypted_pw)
```

A constant shift key and the creation of new input list (Variable: Keyboard).

This section covers the whole encryption process where it will match each password character with the shifted Keyboard to produce a final encrypted password.

This is the encryption method I have discussed in the analysis stage, where a new input list (which is **Keyboard** variable) will be created by shifting the original input list by a number which is shift key. I used 9 as the shift key, so that I could decrypt the encrypted password using one key.

Decryption:

```
33 - def decryption(encrypted_password):
34     symbols = "!@#$$%^&*()-_+=[]{};:,.<>?"
35     Keyboard = string.ascii_uppercase + string.ascii_lowercase + string.digits + symbols
36
37     # shift key to decrypt the password
38     shift_key = 9
39
40     # Create shifted keyboard
41     shifted = Keyboard[shift_key:] + Keyboard[:shift_key]
42
43     # Decrypt the password
44     decrypted = []
45     for char in encrypted_pw:
46         index = shifted.find(char)
47         decrypted.append(Keyboard[index])
48
49     # Join the Decrypted array to a one string
50     decrypted_pw= ""
51     for i in range(0, len(decrypted)):
52         decrypted_pw = decrypted_pw + decrypted[i]
53     return decrypted pw
54
55 print("Decrypted= ",decryption(encrypted_pw))
56
```

Used **find** library in python to match the password character with the shifted Keyboard

Again, this is the whole decrypting process. This uses the same shifted keyboard to decrypt by matching the encrypted characters to the actual keyboard.

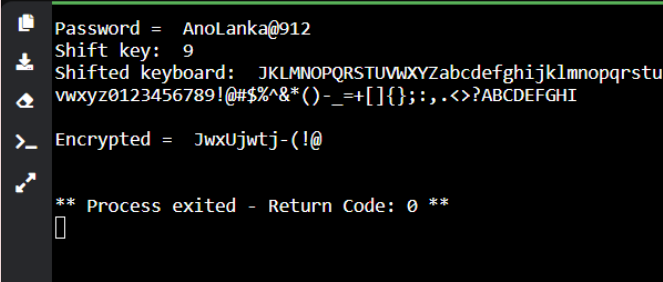
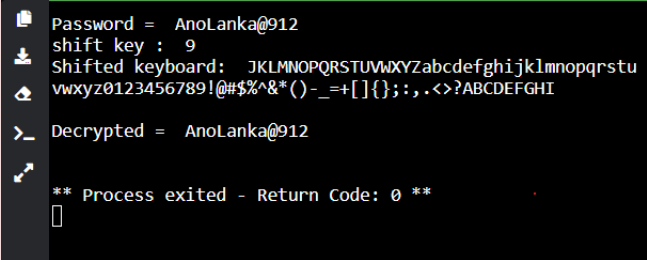
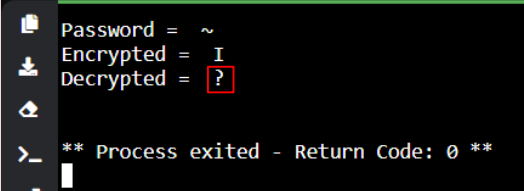
Using 9 as the shift key, I reversed the encryption process by creating the shifted Keyboard and matching the characters in the encrypted password to the actual input list (Keyboard variable).

Output evidence:

```
Run Share $ Command Line Argume
Password = ElonMusk@SpaceX
Encrypted = MtwvU20s)axikmf
Decrypted= ElonMusk@SpaceX
** Process exited - Return Code: 0 **
```

This prototype passes the **black box testing** as it encrypts the password as expected (for example, E must be M when count up 9 in the alphabet). And also decrypts the encrypted password to the actual password.

White box testing:

No	Test Purpose	Evidence	Success
1	Check if the shift key is 9		Pass
2	Check if the Keyboard is shifted by 9		Pass
3	Check if the characters are shifted by 9 in encryption		Pass
4	Check if the decryption key is 9		Pass
5	Check if the decryption Keyboard is shifted by 9		Pass
6	Check if the characters are shifted by 9 in decryption		Pass
7	Check if character outside the keyboard gives “error”	 [No error handling done in the code]	Fail

Justification:

According to the above output evidence, the code passes the black box testing as it outputs the expected outcome. However, there is an issue with the logic used, using the same key for every password is less secure as if a hacker found the shift key (which is 9), he can unlock all the passwords easily.

Therefore, I decided to use random library to generate random number as the shift key. And thinking about decryption, I need the key for each password in the process of decryption. Should be stored somewhere in the database. Therefore, I decided to append the shift key to the end of the encrypted password, rather than storing the key in a separate database table, so that the encryption key can be popped from the encrypted password and use to decrypt.

Review:

This prototype is same as the encryption that was discussed in the analysis stage. I successfully build the prototype 1, but failed to error handle since I haven't code for any inputs beyond expected. Therefore, I decided to error handle for the final prototype I will be creating, and for now optimize the logic of the encryption until I get a more secure encryption method compared to this prototype. Hence, I will be optimizing a second prototype by using random libraries to generate different keys for every password user save.

References:

1. **YouTube Video:** (NeuralNine, Simple Caesar Encryption in Python, 2021)
2. **YouTube Video:** (BroCode, Encryption program in Python , 2022)
3. **Website W3school:** (W3 school, 2025) [Used to understand how functions work]

2/12/2025

Prototype 2

Encryption:

```

main.py  +
1 import random
2 import string
3
4 password = "ElonMusk@SpaceX"
5 print("Password = ",password)
6 def encryption(password):
7     symbols = "!@#$%^&*()-_+[]{};:.,<>?"
8     # every character a user can input
9     Keyboard = string.ascii_uppercase + string.ascii_lowercase + string.digits + symbols
10
11     #generates a random number for encryption key
12     shift_key = random.randrange(10,80)
13
14     # creating a new shifted keyboard variable
15     shifted = Keyboard[shift_key:] + Keyboard[:shift_key]
16
17     # Encryptes the password
18     encrypted = []
19     for char in password:
20         index = Keyboard.find(char)
21         encrypted.append(shifted[index])
22         encrypted.append(str(shift_key))
23
24     # Join the encrypted array to a one string
25     Encrypted_password = ""
26     for i in range(0, len(encrypted)):
27         Encrypted_password = Encrypted_password + encrypted[i]
28     return Encrypted_password
29
30 encrypted_pw = encryption(password)
31 print("Encrypted = ",encrypted_pw)

```

To randomly generate 2-digit number so that shift key gives different keys for every password. The keys might reuse, but still it secures compared to constant key

Cast the shift key to string and attach it at the end of the encrypted password.

I used random range between two digit numbers, so that I can build a decryption code by removing the last two characters from the encrypted password as the shift key. Why I didn't use random range from 0 to 80? Because, it would generate 1-digit number which will be impossible to get the correct key back.

Decryption:

```
34 def decryption(encrypted_password):
35     symbols = "!@#$$%^&*()-_+=[]{};,:.<>?"
36     Keyboard = string.ascii_uppercase + string.ascii_lowercase + string.digits + symbols
37
38     # shift key is the last two characters in the encrypted password
39     shift_key = int(encrypted_password[-2:])
40
41     # Remove the last two characters from encrypted password
42     encrypted_text = encrypted_password[:-2]
43
44     # Create shifted keyboard
45     shifted = Keyboard[shift_key:] + Keyboard[:shift_key]
46
47     # Decrypt the password
48     decrypted = []
49     for char in encrypted_text:
50         index = shifted.find(char)
51         decrypted.append(Keyboard[index])
52
53     # Join the Decrypted array to a one string
54     decrypted_pw = ""
55     for i in range(0, len(decrypted)):
56         decrypted_pw = decrypted_pw + decrypted[i]
57     return decrypted_pw
58
59 print("Decrypted= ", decryption(encrypted_pw))
```

The last two digits will be taken as the shift key, and the **excrpyted_text** will be the encrypted password without the key.

Result evidence:

```
Run Share $ Command Line Arguments
Password = ElonMusk@SpaceX
Encrypted = i^(*q[=%Gw)468130
Decrypted= ElonMusk@SpaceX

** Process exited - Return Code: 0 **
```

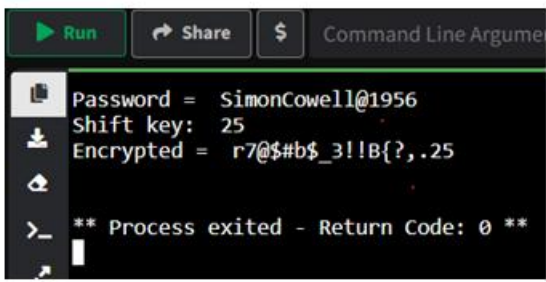
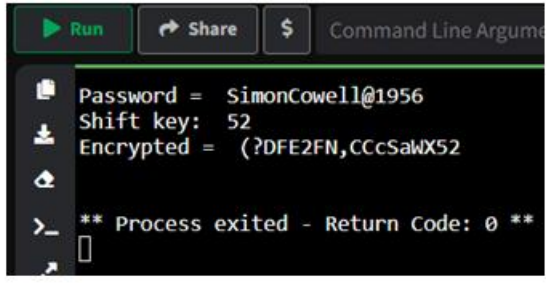
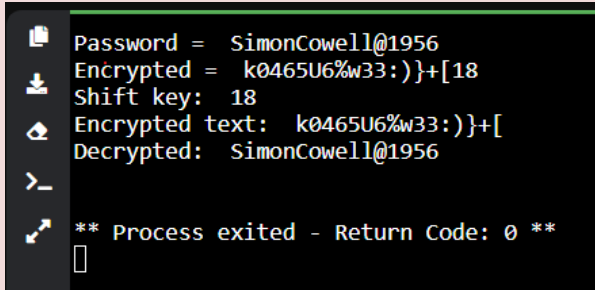
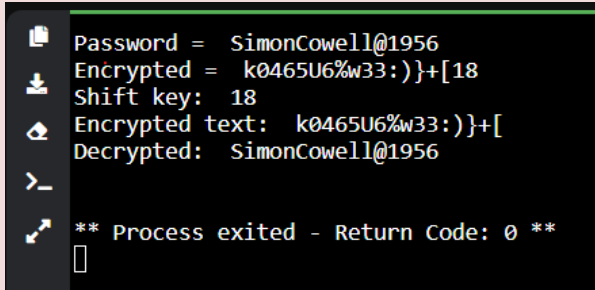
This is the shift key which was randomly generated and appended at the end of encrypted password.

Weaknesses in this prototype:

- 1) Attaching the key to the end of the encryption key is a solution to decrypt easily, but it also rises the security issues with this approach. I am using this encryption method to show my programming skill and to emphasise how encryption was born. I would use high level encryption such as AES or RSA if this was a real password vault.

- 2) The encrypted password contains same type alternative characters. Since Keyboard variable is `string.ascii_uppercase + string.ascii_lowercase + string.digits + symbols`, the chance of having the same character type next to each character is high. For example, according to the results evidence above, there are 3 consecutive symbols next to **i**. This could make the encryption weak.

White box testing

No	Purpose	Evidence	success
1	Does the shift key produce random numbers?		Pass
2	Does the shift key get attached to the encrypted password		Pass
3	Does the decryption process take the shift key correctly?		Pass
4	Is the shift key removed from the encrypted password?		Pass

7/12/2025

New approach with modified Caesar cipher:

To tackle the security, issue of attaching the key with encrypted password, I came up with a new modified version of Caesar cipher. A way where the key is attached at the end but hidden without directly attaching the actual key.

Caesar Cipher Version 2: (logic)

Encryption:

Password:

ElonMusk@1938!

Shift key:

Length of password (14)

Encrypt:

Encrypted password + first character of the actual password (E)

Decryption:

Encrypted password:

Svrk@finc83baE

This is basically
the keyPosition of the last character
in encrypted password in the
actual Keyboard:

(-)

Position of the first character
in the encrypted password in
the actual Keyboard:

Lets say 30

(-)

Then this should be 16



= Shift key: Length of password (14)

I will also hardcode the Keyboard variable. I will first shuffle the whole 87 characters using my shuffle function I built in the Auto Password generation prototype and then use the shuffle keyboard in the Keyboard variable to reduce the chance of similar alternative character types.

Also I decided to add 4 random character from the Keyboard variable to the end of encrypted password to increase encryption strength so that the first character of the actual password is hidden within the encrypted password and the last characters are meaningless.

8/12/2025

Prototype 3

Shuffle of the Keyboard variable:

```

main.py +
1 import string
2 import random
3
4 # Shuffle code
5 symbols = "!@#%&*()-_+[]{};:.,<>?"
6 Keyboard = string.ascii_uppercase + string.ascii_lowercase + string.digits + symbols
7
8 shuffled = []
9 while len(Keyboard) > 0:
10     rand_index = random.randrange(0, len(Keyboard))
11     shuffled.append(Keyboard[rand_index])
12     Keyboard = Keyboard[:rand_index] + Keyboard[rand_index+1:]
13
14 # Convert list back to string
15 NewKeyboard = ""
16 for char in shuffled:
17     NewKeyboard += char
18
19 print(NewKeyboard)
20

```

Run Share Command Line Arguments

(l@p?sil3=g0Z5>2-TBUzx]7tGVHw8&.[4_n)F{09^ENKhD6v:f\$XdPeaAq1yYcMI,JoJmK)*S%#u<bcRrLQ;W+

** Process exited - Return Code: 0 **

Encryption:

```

1 import random
2 import string
3
4 password = "Cristiano@1984"
5 print("Password : ",password)
6
7 def encryption(password):
8     # hardcoded all user inputs
9     Keyboard = "(l@p?sil3=g0Z5>2-TBUzx]7tGVHw8&.[4_n)F{09^ENKhD6v:f$XdPeaAq1yYcMI,JoJmK)*S%#u<bcRrLQ;W+"
10    shift_key = len(password) # shift key
11
12    # Shifting the whole keyboard
13    shifted = Keyboard[shift_key:] + Keyboard[:shift_key]
14
15    encrypted = []
16    for char in password:
17        index = Keyboard.find(char)
18        encrypted.append(shifted[index])
19    encrypted.append(str(password[0]))
20
21    #Adding 4 random characters from keyboard to the encrypted password
22    for i in range(4):
23        Temp = random.randrange(1,86) # temporary holds a random number
24        encrypted.append(str(Keyboard[Temp]))
25
26    return "".join(encrypted)
27
28 encrypted_pw = encryption(password)
29 print("Encryption: ",encrypted_pw)

```

This is the shuffled keyboard which was shown above

Attaching 4 random characters to the end of the encryption using a for loop. **Temp** variable hold the random number.

As discussed in the new Caesar cipher version, the encryption is logical optimized. The shift key is the length of the password and once encrypted, 4 random character are added at the end of the encrypted password. When decrypting, the 4 last characters are removed and the new last character will be used to decrypt the encrypted password.

Decryption:

```

31 - def decryption(encrypted_password):
32
33     length = len(encrypted_password)
34     Keyboard = "(l@p?si!3-g0Z5>2-TBUzx]7tGVHw8&. [4_n]F{09^ENKhDGv:f$XdPeaAq1yYcMI,JoJMk)*5%#u<bcRrLQ;W+!"
35
36     encrypted_text=encrypted_password[:-4] # Removal of unnecessary last 4 characters
37
38     # linear search function to find position
39 - def linear_search(char):
40     find = str(char)
41     for i in range(len(Keyboard)):
42         if Keyboard[i] == find:
43             position = i+1
44             break
45     return position
46
47     first_char = encrypted_text[-1] # first character of the actual password
48     FCA = linear_search(first_char) # position of first character of the actual password
49     FCE = linear_search(encrypted_text[0]) # position of first character of encrypted password
50
51     shift_key=FCE-FCA # Difference gives the shift key (length of actual password)
52
53     shifted = Keyboard[shift_key:] + Keyboard[:shift_key]
54
55     decrypted = []
56
57 - for char in encrypted_text:
58     index = shifted.find(char)
59     decrypted.append(Keyboard[index])
60     decrypted.pop() # remove the first character of the actual password
61
62     return "".join(decrypted)
63
64 decrypted_text= decryption(encrypted_pw)
65 print("Decryption: ",decrypted_text)
66

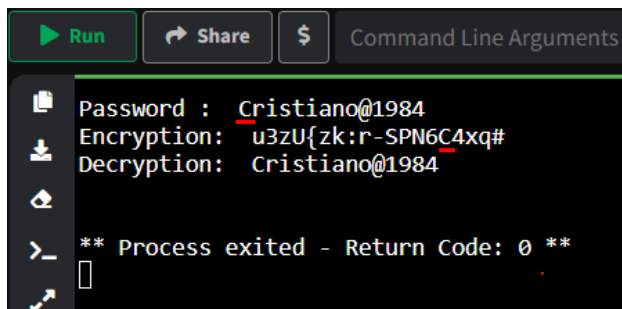
```

Firstly, the unnecessary last 4 characters is removed as shown. Linear function used to search the position of first and last characters.

Shift key is the difference between FCA and FCE as we discussed in the logic explanation

I decided to create a function for searching because, the searching is needed in two variables and so can be called rather than re writing the same set of code twice. Also, linear search is better than binary search because the maximum loop is 87 times (not too long) and the list does not need to be sorted (binary search is impossible to use in this scenario).

Output evidence:



```

Run Share $ Command Line Arguments
Password : Cristiano@1984
Encryption: u3zU{zk:r-SPN6C4xq#
Decryption: Cristiano@1984

** Process exited - Return Code: 0 **

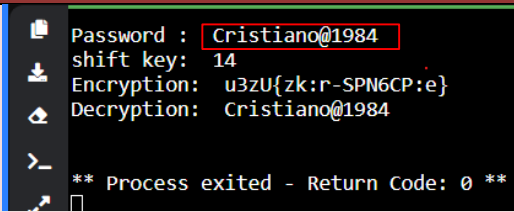
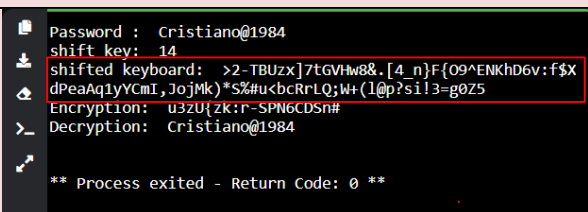
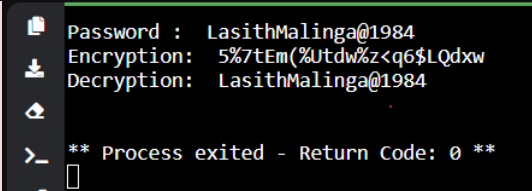
```

The first character of the actual password is hidden before last 4 characters as shown in the output evidence. The code decrypts the actual password as shown in the evidence

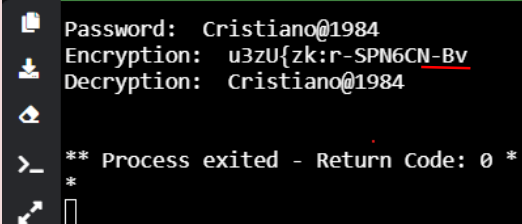
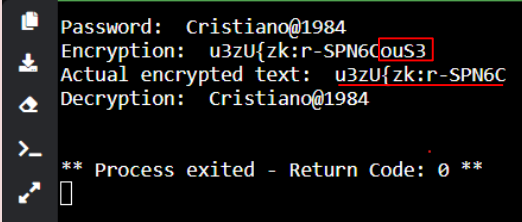
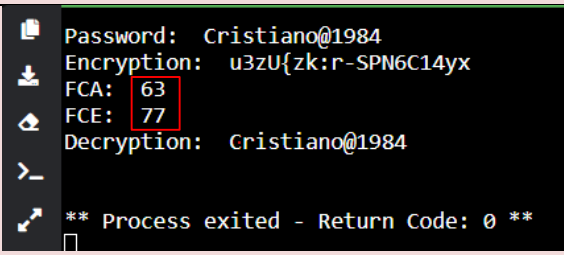
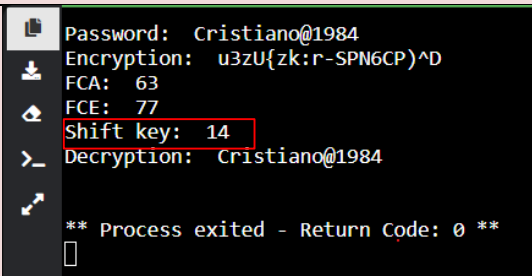
Justification:

Since, this prototype solves the main issues as shown in the first 2 prototypes and is logically improved, I decided to leave this prototype as the final prototype. And so, the next stage after the final prototype as discussed is to develop error handling.

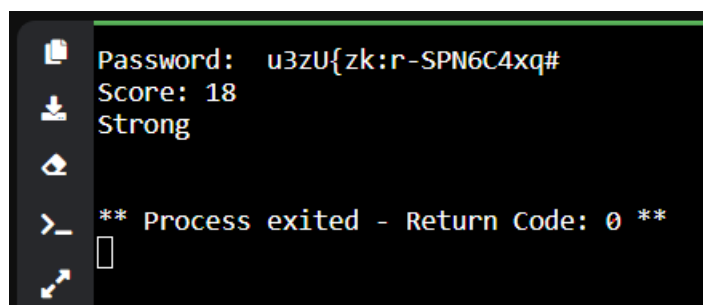
White box testing:

No	Purpose	Evidence	Success
1	Check if the shift key id length of the password	 <pre> Password : Cristiano@1984 shift key: 14 Encryption: u3zU{zk:r-SPN6CP:e} Decryption: Cristiano@1984 ** Process exited - Return Code: 0 ** </pre>	pass
2	Check if keyboard is shifted by 14 (therefore, the first character should be > according to the hardcoded input list)	 <pre> Password : Cristiano@1984 shift key: 14 shifted keyboard: >2-TBUzx]7tGVHw8&.[4_n}F{09^ENKhD6v:f\$X dPeaAqlyYCmI,JoJmK)*S%#u<bcRrLQ;w+(l@p?sil3=g0Z5 Encryption: u3zU{zk:r-SPN6CDSn# Decryption: Cristiano@1984 ** Process exited - Return Code: 0 ** </pre>	pass
3	Check if the first character of the actual password is attached to the encrypted password	 <pre> Password : LasithMalinga@1984 Encryption: 5%7tEm(%Utdw%z<q6\$LQdxw Decryption: LasithMalinga@1984 ** Process exited - Return Code: 0 ** </pre>	pass

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

4	Check if there are 4 random characters at the end of the encrypted password	 <pre> Password: Cristiano@1984 Encryption: u3zU{zk:r-SPN6CN-BV Decryption: Cristiano@1984 ** Process exited - Return Code: 0 ** </pre>	Pass
5	Check if the last 4 unnecessary characters are removed	 <pre> Password: Cristiano@1984 Encryption: u3zU{zk:r-SPN6C<u>ouS3</u> Actual encrypted text: u3zU{zk:r-SPN6C Decryption: Cristiano@1984 ** Process exited - Return Code: 0 ** </pre>	Pass
6	Check if the linear search algorithm is correct.[If linear search algorithm is correct, FCA should be 63 and FCE should be 77]	 <pre> Password: Cristiano@1984 Encryption: u3zU{zk:r-SPN6C14yx FCA: 63 FCE: 77 Decryption: Cristiano@1984 ** Process exited - Return Code: 0 ** </pre>	Pass
7	Check if the shift key is correct. [The shift key should be 14]	 <pre> Password: Cristiano@1984 Encryption: u3zU{zk:r-SPN6CP)^D FCA: 63 FCE: 77 Shift key: 14 Decryption: Cristiano@1984 ** Process exited - Return Code: 0 ** </pre>	Pass

Test the strength using my strength checker:



```

Password: u3zU{zk:r-SPN6C4xq#
Score: 18
Strong

** Process exited - Return Code: 0 **

```

According to the password strength checker I build, this password scores the highest possible score which is 18 with a strength status of “strong”. This shows that the encrypted password are strong and random.

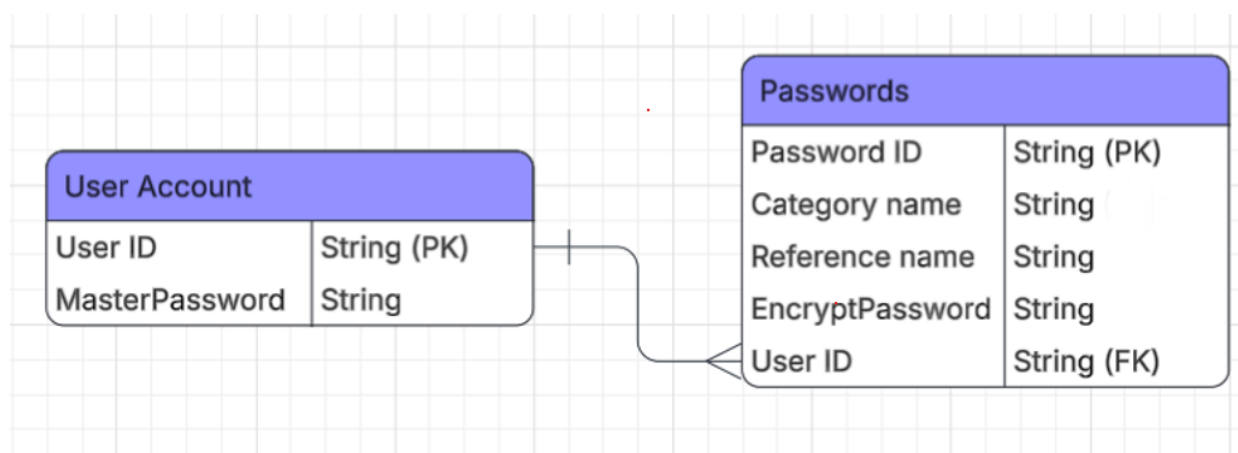
Review:

In this prototype, I built the final modified Caesar cipher model that I discussed after prototype 2. I decided to make this prototype the final encryption and decryption product as it covers all the main security features and passed all the testing including: white box testing, black box testing and strength test. The next task is to error handle the final prototype and finalize the encryption module in the password vault when connecting the frontend and backend development.

13/12/2025

DEVELOPMENT OF DATABASE

I chose to develop the database using pycharm SQLite library instead of connecting an external SQL database such as MySQL. The main reason for this is my project does not hold thousands of passwords as it is only a demonstration of my project work, and using an internal library is more flexible and easy to develop. As discussed in the design below images is the design of the database:



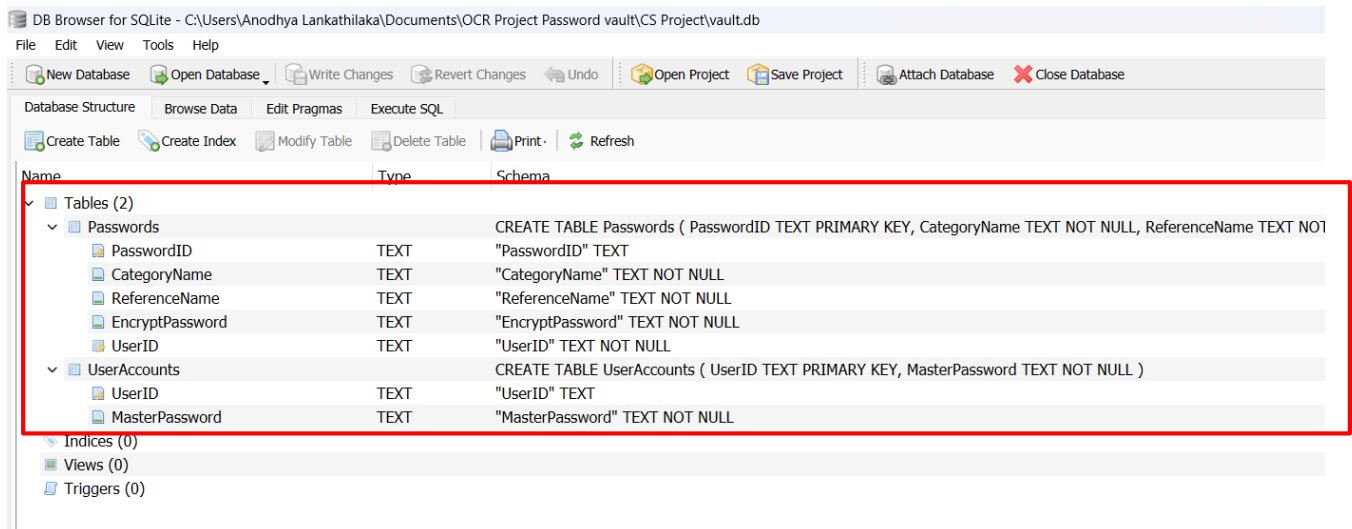
Creation of two tables: [Import SQLite]

```
18  ~ # Creation of database table
19      # Connect to the database and create a file (vault.db)
20      conn = sqlite3.connect("vault.db")
21      # Enable foreign keys
22      conn.execute("PRAGMA foreign_keys = ON")
23      cursor = conn.cursor()
24
25      # UserAccount table
26      ~ cursor.execute("""
27          CREATE TABLE IF NOT EXISTS UserAccounts (
28              UserID TEXT PRIMARY KEY,
29              MasterPassword TEXT NOT NULL
30          );
31      """)
32
33      # Passwords table
34
35      cursor.execute("""
36          CREATE TABLE IF NOT EXISTS Passwords (
37              PasswordID TEXT PRIMARY KEY,
38              CategoryName TEXT NOT NULL,
39              ReferenceName TEXT NOT NULL,
40              EncryptPassword TEXT NOT NULL,
41              UserID TEXT NOT NULL,
42              FOREIGN KEY (UserID) REFERENCES UserAccount(UserID)
43          );
44      """)
45      # save and close database
46      conn.commit()
47      conn.close()
```

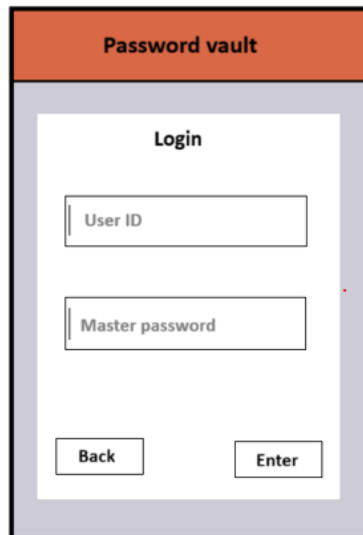
SQLite ignores foreign keys by default, meaning the foreign key connect is OFF unless we turn it ON. This line of code enables the connection and between the two tables

This saves the database tables and close the connection between SQLite. If we don't close the table, the database might get locked when we recall the database later for other function.

I downloaded a database viewer to view the database I created so that it clearly shows the tables and the data stored. Below is an evidence of successfully creating the database tables:



Login page: [Import messagebox, Import hashlib, Import sqlite 3]



Firstly, the database will be connected to proceed the main purpose of checking if such user exists.

Once user enter fill the input boxes and **click Enter button**:

1. Get the User ID
2. Get the Master password
3. Hash the Master password
4. Check if User ID exists
5. Check if hash password exists
6. If both condition successful take user to the account
7. Else, output the error message

```
117 # Process once user click the enter button
118 def enter():
119     user_id = username_entry.get()
120     masterpassword = password_entry.get()
121
122     def hash_password(password):
123         return hashlib.sha256(password.encode()).hexdigest()
124
125     conn = sqlite3.connect("vault.db")
126     conn.execute("PRAGMA foreign_keys = ON")
127     cursor = conn.cursor()
128
129     cursor.execute(_sql: "SELECT MasterPassword FROM UserAccounts WHERE UserID = ?", _parameters: (user_id,))
130     result = cursor.fetchone()
131     conn.close()
132
133     if result is None:
134         messagebox.showerror(title="Login Failed", message="User ID not found.")
135         return
136
137     stored_hash = result[0]
138     entered_hash = hash_password(masterpassword)
139
140     if entered_hash == stored_hash:
141         messagebox.showinfo(title="Login", message="Successfully logged in")
142     else:
143         messagebox.showerror(title="Login Failed", message="Incorrect password.")
144
145     Button(self.inner, text="Enter", font=FONT_BUTTON, width=10, command=enter).pack(side="right", pady=45, padx=20)
```

This section of lines connects the database and the foreign key to get stored data in vault.db

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

Since the test cannot be run without accounts created, I will develop the create account page database and then run a test for both the pages at once. For now, the login page database only output “User ID not found” as expected.

Create account page: [Import string, Import messagebox, Import hashlib, Import sqlite 3]



Firstly, a validation function will be built to check if master password satisfied the requirements. Next the database will be connected to save the new account once Save button is clicked.

1. Get User ID
2. Get Master Password
3. Check if User ID already exists
4. Hash the Master password
5. Check if validation function returns True
6. Save the account in the database

Validation code:

```
187       # Validation test function
188       def validate_password(password):
189           upper = string.ascii_uppercase
190           symbols = "!@#$%^&*()-_+=[]{};:,<.>?"
191
192           # linear search
193           def linear_search(password, search_type):
194               for i in range(len(password)):
195                   for j in range(len(search_type)):
196                       if password[i] == search_type[j]:
197                           return "Found"
198
199           # search Uppercase letter and symbols
200
201           Upper_exist =linear_search(password, upper)
202           Symbol_exist =linear_search(password, symbols)
203           if len(password) < 8 or Upper_exist != "Found" or Symbol_exist != "Found":
204               return False
205           return True
```

A function that will use linear searching to check if user entered at least 1 Uppercase and special character. I used a function to reduce repetition in code.

Check if the user met all three validation requirements using OR logic

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>

Database code:

```
207 def save():
208     # get user inputs
209     user_id = username_entry.get()
210     password = password_entry.get()
211
212     if not validate_password(password):
213         messagebox.showerror(title: "Error", message: "Password does not meet the validation rules.")
214         return
215
216     # hash the password
217     hashed_password = hashlib.sha256(password.encode()).hexdigest()
218
219     # Database connection
220     conn = sqlite3.connect("vault.db")
221     conn.execute("PRAGMA foreign_keys = ON")
222     cursor = conn.cursor()
223
224     # Ensure table exists
225     cursor.execute("""
226         CREATE TABLE IF NOT EXISTS UserAccounts (
227             UserID TEXT PRIMARY KEY,
228             MasterPassword TEXT NOT NULL
229         )
230     """)
231
232     # Check if user exists
233     cursor.execute(__sql: "SELECT * FROM UserAccounts WHERE UserID=?", __parameters: (user_id,))
234     if cursor.fetchone() is not None:
235         messagebox.showerror(title: "Error", message: "User ID already exists.")
236         conn.close()
237         return
238
239     # Insert new user
240     cursor.execute(__sql: "INSERT INTO UserAccounts (UserID, MasterPassword) VALUES (?, ?)", __parameters: (user_id,
241     conn.commit()
242     conn.close())
243     messagebox.showinfo(title: "Success", message: "Account created successfully!")
```

Handwritten annotations on the code:

- Arrows 1 and 2 point to `user_id = username_entry.get()` and `password = password_entry.get()` respectively.
- Arrow 3 points to the `__parameters: (user_id,)` in the SELECT query.
- Arrow 4 points to the `hashed_password` variable.
- Arrow 5 points to the `validate_password` function call.
- Arrow 6 points to the `cursor.execute` call for the SELECT query.

Callout box for the database connection section (lines 219-222):

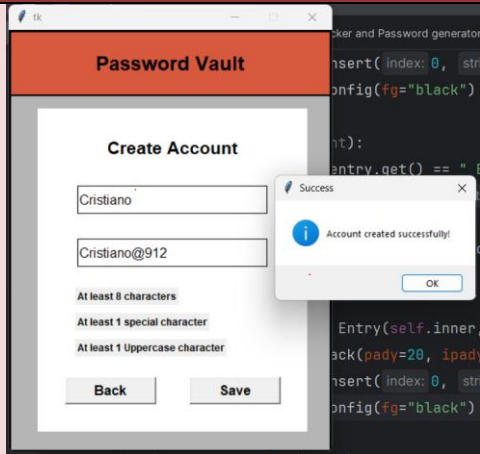
Conn: connects the file to the sqlite database (vault.db), and turns PRAGMA foreign key connection ON to make sure the foreign key is accessed by the Password Table.

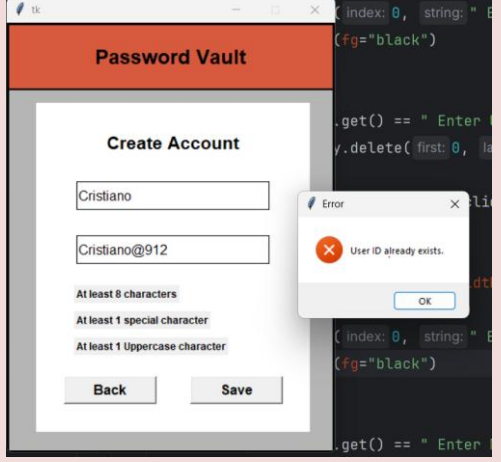
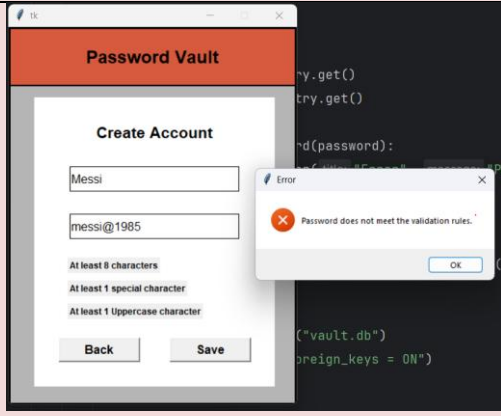
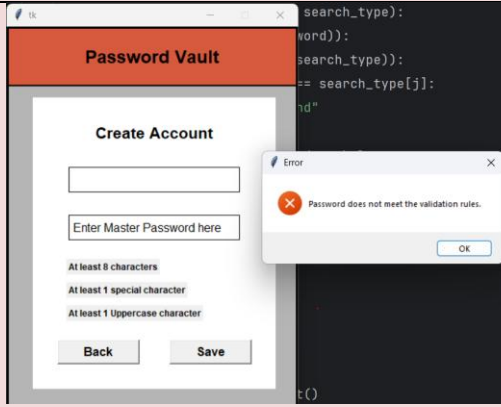
For both Login page and Create account page database coding, I used various different online resources such as SQLite manual documents and multiple YouTube videos. All the online resources will be documented below.

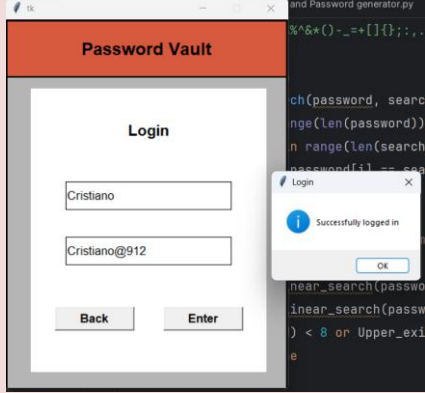
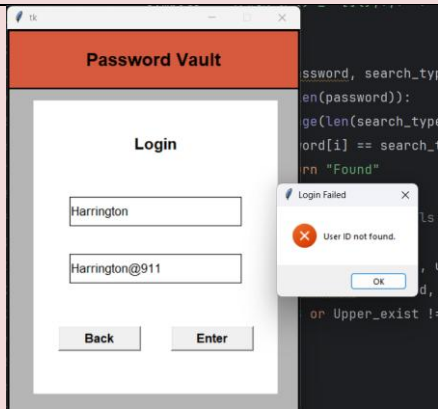
References:

1. **SQLite Foreign key support manual:** (SQLite, 2024) Used this to make sure foreign key is accessed by the all tables by turning ON its connection.
2. **YouTube video:** (Kite, 2021) Used to understand how to create tables, connect tables when using, and save and close the tables when needed.
3. **YouTube video:** (Digital, 2022) Used to understand how to insert user inputs into the database tables
4. **YouTube video:** (CodingUnderPressure, 2023) Used to understand how the database viewer I download works. Database viewer I downloaded was DB Browser (SQLite)
5. **YouTube video:** (FreeCodeCamp.org, 2020) Used to learn how to create database and all the relevant basic coding skills when developing database.

Testing [Login + Create account]:

No	Purpose	Evidence	Success
1	Check if Account creates successfully when met all entry requirements Input: Non existing username Expected outcome: message stating the account creation was successful		Pass

2	Check if User will not be allowed with already existing UserID Input: Cristiano in UserID box Expected outcome: message stating the account already exists	 The screenshot shows a Tkinter window titled 'Password Vault' with a 'Create Account' form. The form has two input fields: the first contains 'Cristiano' and the second contains 'Cristiano@912'. Below the fields are three validation rules: 'At least 8 characters', 'At least 1 special character', and 'At least 1 Uppercase character'. There are 'Back' and 'Save' buttons. An error dialog box is displayed over the form, stating 'User ID already exists.' with an 'OK' button.	Pass
3	Check if user will not be allowed to create account without meeting the validations Input: password with no Uppercase letter Expected outcome: message stating password haven't met the validation requirements	 The screenshot shows the same 'Password Vault' window. The first input field contains 'Messi' and the second contains 'messi@1985'. The error dialog box now states 'Password does not meet the validation rules.' with an 'OK' button.	Pass
4	Check if empty user input boxed would output an error message. I haven't yet coded any error handling for incorrect user inputs except for Master password validation	 The screenshot shows the 'Password Vault' window with empty input fields. The error dialog box still displays 'Password does not meet the validation rules.' with an 'OK' button.	Fail

5	Check if user can log into already existing accounts. Input: Cristiano and Cristiano@912 Expected outcome: Successfully log in to the account		Pass
6	Check if Non existing input details would pop up an error message stating no account found		Pass

Development of further workflow after login:

I found some error I have done in my code architecture (Mainly in the frontend). Even though I have developed the each and every page in Frontend development module, I have not thought about the logical flow of the system. For example, what happens after user log into their accounts.

1. The Switch_page function I have built at the end of every function should be in the Base page function. Duplicating the switch_page will not allow the workflow of the system.
2. UserID should be passed as a parameter so that the pages can connect to their account database using the primary key of User Account tale which is user_id.

Correction of workflow:

- 1) Added the switch page function to the Basepage so that there is no need for duplicating same function. This approach is more logical and provides more flexible architecture.

```
74 75 76 7 usages
def switch_page(self, page_class, *args):
    self.destroy()
    page_class(self.master, *args)
```

Therefore, I removed all the switch_page functions rewritten inside every page functions.

- 2) In login page, I rewrote the messagebox of printing “Logged in” to the switch_page function so that when user enter the correct account details, he will be taken into his account.

```
if entered_hash == stored_hash:
    self.switch_page(Mainpage, *args: user_id)
else:
    messagebox.showerror( title: "Login Failed", message: "Incorrect password.")
```

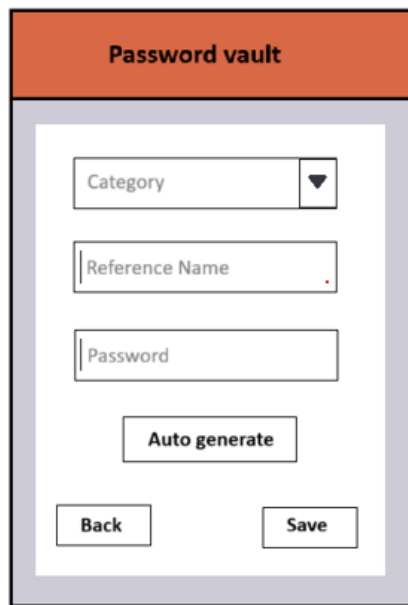
- 3) In Main page, I passed the user_id as a parameter from login page so that the user id can be used to retrieve the data in the database.

```
class Mainpage(BasePage):
    def __init__(self, master, user_id):
        super().__init__(master)
        self.user_id = user_id
        self.card.configure(bg="#B5B5B5")
        self.inner.configure(bg="#B5B5B5")

        #Buttons
        lable=Label(self.inner, text=f"Welcome {user_id}", font=("Arial", 12, "bold"), bg="#B5B5B5")
        lable.place(x=90, y=0)
```

I haven't created a dynamic page function that will change according to what button user click in the Main page. So next step is to build the dynamic page and connect it so that the system will flow as expected. Firstly, I will create the Store password page database and allow user to insert data into the database. After completing this page, I will then start to code the dynamic page (Category_page).

Store Password Page:

A screenshot of a web form titled "Password vault" in an orange header. The form has a light purple border and contains three input fields: "Category" with a dropdown arrow, "Reference Name" with a red dot at the end, and "Password". Below these fields is an "Auto generate" button. At the bottom are two buttons: "Back" and "Save".

Firstly, the encryption and Auto Password generation functions I built in backend development will be included into the code as functions.

- 1) Get user inputs into three variables
- 2) Make sure the password generator will generate passwords as it is clicked
- 3) Encrypt the password
- 4) Connect to database
- 5) Insert the data into the Passwords table

Auto password generator as user click Auto generate button:

```

314     def autofill():
315         # Auto password generator function
316         def AutoGenerate():
317
318             # Each set of characters in a character type is assigned a variable
319
320             upper = string.ascii_uppercase
321             lower = string.ascii_lowercase
322             digits = string.digits
323             symbols = "!@#$%^&*()-_+=[]{};,:.<.>?"
324
325             character_type = [upper, lower, digits, symbols]
326
327             # Creation of password with 4 characters in each character type
328             password_chars = []
329
330             for i in range(0, 4):
331                 for j in range(0, 4):
332                     password_chars.append(random.choice(character_type[i]))
333
334             # Shuffle code for password_chars
335             shuffled = []
336             while len(password_chars) > 0:
337                 rand_index = random.randrange(start=0, len(password_chars))
338                 shuffled.append(password_chars[rand_index])
339                 password_chars.pop(rand_index)
340             password_chars = shuffled
341
342             # Combine the password characters to a string
343
344             password = ""
345             for i in range(0, len(password_chars)):
346                 password = password + password_chars[i]
347             return password
348
349             Autopassword = AutoGenerate()
350
351             SPASSWORD_entry.delete(first=0, last="end")
352             SPASSWORD_entry.insert(index=0, Autopassword)

```

D. EVALUATION

Candidate Name: <INSERT NAME HERE> Candidate Number: <NNNN>
<See H446-03 Project Advice Booklet for help and guidance of what must go here.>

PROJECT APPENDIXES

Insert as many project appendixes as you need for your project.

These might include, but are not limited to:

- Complete Code Listing (ESSENTIAL)
- Interview Transcripts
- Meeting notes
- Observation notes or questionnaires

REFERENCE

About Bitwarden password saving software (Wikipedia, 2025)

Bitwarden Password Manager Weaknesses (Gilbertson, 2025)

About Keepass Password Manager (KeepassXC, 2025)

Keepass Password Manager weaknesses (Parvez, 2025)

Enpass Password Managing software (EnPass, 2024)

KeeWeb Password Managing software (KeeWeb, 2025)